

**A MAJOR PROJECT REPORT
ON**

“KeyDash”

**Submitted for the partial fulfillment of the Degree of Bachelor of Engineering in
Computer awarded by Pokhara University**



SUBMITTED BY:

Britant Shrestha (22070449)

Manish Shrestha (22070452)

Ranjana Dhakal (22070457)

Rijan Shrestha (22070458)

SUPERVISOR:

Asst. Prof. Er. Shiva Kunwar

Affiliated to Pokhara University

**DEPARTMENT OF COMPUTER ENGINEERING
POKHARA ENGINEERING COLLEGE**

Phirke, Pokhara-8

2026

CERTIFICATE

Date: 2026/07/24

This is to certify that the project entitled “**KeyDash**” has been carried out by Britant Shrestha (22070449), Manish Shrestha (22070452), Ranjana Dhakal (22070457) and Rijan Shrestha (22070458) in partial fulfillment of the degree of Bachelor of Computer Engineering of Pokhara University, during the academic year 2026. To the best of our knowledge and belief, this work has not been submitted elsewhere for the award of any other degree and thus has been accepted.

Asst. Prof. Er. Shiva Kunwar
Supervisor

Asst. Prof. Er. Krishna Ghimire
Co-Ordinator, Research Management Cell

Asst. Prof. Er. Suraj Basant Tulachan
HOD, Computer & Electronics

Asst. Prof. Dr. Bidur Devkota
External Examiner

Acknowledgement

We would like to extend our heartfelt gratitude to everyone who supported and guided us throughout the completion of our project. First and foremost, we are profoundly thankful to our respected supervisor, **Er. Shiva Kunwar**, for his expert guidance, continuous encouragement, and constructive suggestions at every stage of this project. His patience and dedication to our academic growth significantly shaped both the technical and practical direction of our work.

We also express our sincere appreciation to the **Department of Computer Engineering** for providing a strong academic foundation and the essential resources that made this project achievable. The department's curriculum, lab facilities, and regular project evaluations were instrumental in our learning process.

We would like to acknowledge the contributions of all faculty members and technical staff, whose teachings and timely assistance helped us overcome many challenges during the development phase. Their expertise and support added great value to our learning experience.

Special thanks to our classmates and friends who helped with early testing, shared ideas, and gave us constructive feedback that improved the overall quality of our project.

Lastly, we want to recognize the collective effort of our group. This project would not have been possible without our collaboration, mutual understanding, and commitment toward achieving a common goal.

Britant Shrestha (22070449)

Manish Shrestha (22070452)

Ranjana Dhakal (22070457)

Rijan Shrestha (22070458)

Table of Contents

Acknowledgement.....	i
List of Figures.....	vii
List of Tables.....	viii
List of Abbreviations.....	x
Abstract.....	xi
सार.....	xii
Chapter 1 Introduction.....	1
1.1 Background.....	1
1.2 Statement of the Problem.....	1
1.3 Objective.....	2
1.4 Scope of the Project.....	2
1.5 Limitations.....	3
Chapter 2: Literature Review.....	4
2.1 Theoretical Framework & Background.....	4
2.2 Review of Existing Platforms.....	5
2.3 Feature Comparative Analysis.....	5
2.4 Research Gap & Proposed Solution.....	6
Chapter 3: Analysis.....	8
3.1 Project Management.....	8
3.1.1 Initial Investigation.....	8
3.1.2 Feasibility Analysis.....	8
3.1.3 Cost Benefit Analysis.....	9
3.2 Requirement Analysis.....	9
Chapter 4: Design.....	11
4.1 Software Requirements Specification.....	11
4.1.1 Functional Requirements.....	11
4.1.2 System Specification.....	11
4.1.3 Non-functional Requirements.....	11

4.2 Risk Assessment.....	12
Chapter 5: System Modeling.....	14
5.1 Block Diagram.....	14
5.2 UML Diagram.....	15
5.2.1 UML Use Case Diagram.....	15
5.3 Flowcharts.....	16
5.3.1 System Flowchart.....	16
5.4 System Workflow.....	17
5.4.1 Registering Users.....	18
5.4.2 Log In Option.....	18
5.4.3 View modes and respective levels.....	18
5.4.4 View Leaderboard.....	18
5.4.5 View Scoreboard.....	18
5.4.6 Profile Section.....	19
5.4.7 Multiplayer Race.....	19
5.4.8 Battle with CPU.....	19
5.5 Data Flow Diagram.....	20
5.5.1 Levels of DFD.....	20
5.6 Database Design.....	21
5.6.1 ER (Entity Relationship) Diagram.....	21
5.6.2 Supabase Database Table for users.....	22
5.6.3 Supabase As a Database.....	35
Chapter 6: Coding.....	37
6.1 Hardware Specification.....	37
6.2 Platform.....	37
6.3 Technology Stack and System Theory.....	37
6.3.1 React Library.....	37
6.3.2 Supabase Backend.....	37
6.3.3 PostgreSQL Database.....	38

6.3.4 User Authentication.....	38
6.3.5 Relational Data Modeling.....	38
6.3.6 Frontend State Management.....	38
6.3.7 Version Control & CI/CD Pipelines.....	39
6.3.8 Application Security Considerations.....	39
6.4 System Classification & Application Category.....	39
6.5 Programming Language Used.....	40
6.5.1 JavaScript Programming Language.....	40
6.5.2 SQL (Structured Query Language).....	40
6.6 Software Tools Used.....	41
6.6.1 Visual Studio Code.....	41
6.6.2 Git and GitHub.....	41
6.6.3 Web browser.....	41
6.6.4 Adobe Illustrator.....	42
6.7 Coding Style Followed.....	42
6.8 Core Logic & Code Implementation.....	43
6.8.1 WPM & Accuracy Calculation Engine.....	43
6.8.2 Caret Tracking & Input Synchronization.....	43
6.8.3 Real-Time Keystroke & Delay Telemetry Engine.....	44
6.8.4 Performance Recommendation Engine.....	45
Chapter 7: Testing.....	47
7.1 Test Plan.....	47
7.2 Test Cases.....	47
7.2.1 Testing Cases Summary.....	47
7.2.2 Sample Test Cases.....	48
Chapter 8: Conclusion and Future Work.....	55
8.1 Future Work.....	55
8.2 Conclusion.....	55
REFERENCES.....	57

APPENDICES

Appendix A: Login Screen.....	58
Appendix B: Sentence Mode.....	58
Appendix C: Time Mode.....	59
Appendix D: Leaderboard.....	59
Appendix E: Profile Section.....	60
Appendix F: Statistics Section.....	60
Appendix G: Profile Update.....	61
Appendix H: Database Section.....	62
Appendix I: Daily Challenge.....	62
Appendix J: Battle with CPU.....	62
Appendix K: Coding Mode.....	63
Appendix L: Symbol Mode.....	64
Appendix M: Custom Mode.....	64
Appendix N: Dashboard Section.....	65
Appendix O: Multiplayer Race Mode.....	65
Appendix P: Multiplayer Race Settings.....	66
Appendix Q: Multiplayer Race Waiting Area.....	66
Appendix R: Multiplayer Race Results.....	66
Appendix S: Nepali Language Sentence Mode.....	67
Appendix T: Nepali Language Timed Mode.....	67
Appendix U: Nepali Language with Roman Keyboard.....	68
Appendix V: Nepali Language with Traditional Keyboard.....	68
Appendix W: Result Page.....	69
Appendix X: Result Page Recommendations and Review.....	69

Appendix Y: Result Page Paragraph Review and Social Sharing.....	70
Appendix Z: Result Page Social Sharing Link Ready.....	70

List Of Figures

S.N.	FIGURE	Page No.
1.	Figure 5.1: Block Diagram of the System.....	14
2.	Figure 5.2: UML Use Case Diagram.....	15
3.	Figure 5.3: System Flowchart.....	16
4.	Figure 5.4: System Workflow.....	17
5.	Figure 5.5: Context Level DFD.....	20
6.	Figure 5.6: Level-1 DFD.....	20
7.	Figure 5.7: ER Diagram.....	21
8.	Figure 6.1: WPM and Accuracy Algorithm.....	43
9.	Figure 6.2: Caret Tracking & Input Synchronization.....	44
10.	Figure 6.3: Real-Time Keystroke & Delay Telemetry Engine.....	45
11.	Figure 6.4: Performance Recommendation Engine.....	46
12.	Figure 7.1: Test case 1.....	48
13.	Figure 7.2: Test case 2.....	49
14.	Figure 7.3: Test case 3.....	50
15.	Figure 7.4: Test case 4.....	51
16.	Figure 7.5: Test case 5.....	52
17.	Figure 7.6: Test case 6.....	53
18.	Figure 7.7: Test case 7.....	54

List Of Tables

S.N.	TABLE	Page no.
1.	Table 2.1: KeyDash vs. Existing Typing Assessment Platforms.....	5
2.	Table 5.1: Supabase Table profile.....	23
3.	Table 5.2: Supabase Table leaderboard_sentence.....	24
4.	Table 5.3: Supabase Table leaderboard_timed.....	24
5.	Table 5.4: Supabase Table typing_history.....	26
6.	Table 5.5: Supabase Table challenges.....	26
7.	Table 5.6: Supabase Table challenge_attempts.....	27
8.	Table 5.7: Supabase Table leaderboard_daily.....	28
9.	Table 5.8: Supabase Table leaderboard_weekly.....	28
10.	Table 5.9: Supabase Table leaderboard_monthly.....	29
11.	Table 5.10: Supabase Table multiplayer_rooms.....	30
12.	Table 5.11: Supabase Table multiplayer_participants.....	31
13.	Table 5.12: Supabase Table ai_battles.....	32
14.	Table 5.13: Supabase Table achievements.....	33
15.	Table 5.14: Supabase Table user_achievements.....	33
16.	Table 5.15: Supabase Table custom_texts.....	34
17.	Table 5.16: Supabase Table shared_results.....	35
18.	Table 5.17: Supabase Table multiplayer_invitations.....	35
19.	Table 7.1: Testing Cases Summary.....	47
20.	Table 7.2: Test Case 1.....	48
21.	Table 7.3: Test Case 2.....	49
22.	Table 7.4: Test Case 3.....	50

23.	Table 7.5: Test Case 4.....	51
24.	Table 7.6: Test Case 5.....	52
25.	Table 7.7: Test Case 6.....	53
26.	Table 7.8: Test Case 7.....	54

List of Abbreviations

ACID: Atomicity, Consistency, Isolation, Durability

API: Application Programming Interface

BaaS: Backend-as-a-Service

CI/CD: Continuous Integration and Continuous Deployment

CPU: Central Processing Unit

CSS: Cascading Style Sheets

DFD: Data Flow Diagram

DOM: Document Object Model

ER: Entity Relationship

FK: Foreign Key

HTML: HyperText Markup Language

JSON: JavaScript Object Notation

JSX: JavaScript XML

OTP: One-Time Password

PK: Primary Key

RBAC: Role-Based Access Control

REST: Representational State Transfer

SQL: Structured Query Language

WPM: Words Per Minute

UI: User Interface

UML: Unified Modeling Language

UX: User Experience

ABSTRACT

“**KeyDash**” provides an engaging platform for users to test and improve their typing skills across multiple modes across language mode (Nepali/English), Coding mode, Symbols Mode, and a custom mode for user-supplied text. The application features real-time performance metrics (Words Per Minute - WPM, accuracy, and error count), daily streaks system to build consistent practice habits, and over 1200+ tiered achievements across speed, streaks, coding, symbols, multiplayer and CPU battle categories. Competitive features include daily, weekly, monthly, and all time leaderboards, real-time multiplayer typing races powered by Supabase Realtime and CPU opponents at Easy, medium, Hard and Pro Difficulty. Users can also generate and share result cards on social media. It is built using modern web technologies and it ensures a seamless and responsive experiences across devices. The project emphasizes clean code practices, efficient algorithms for performance calculation, and an intuitive UI/UX design. It serves as both an educational tool, demonstrating proficiency in full-stack web development.

Keywords: *Typing Speed Tester / Typing Assessment, Words Per Minute (WPM) & Accuracy Metrics, Full-Stack Web Development, React.js & Component-Based Architecture, Supabase & Backend-as-a-Service (BaaS), PostgreSQL Database, Real-time Analytics & Synchronization, Multi-language Support (Nepali / English), Gamification (Daily Streaks, Tiered Achievements, Leaderboards), Multiplayer Typing Races & CPU Battles*

सार

“KeyDash” ले प्रयोगकर्ताहरूलाई भाषा मोड (नेपाली/अंग्रेजी), कोडिङ मोड, सिम्बोल (प्रतीक) मोड, र प्रयोगकर्ता-द्वारा दिइएको आफ्नै टेक्स्ट (Custom mode) गरी विभिन्न मोडहरू मार्फत आफ्नो टाइपिङ क्षमता परीक्षण तथा सुधार गर्न एउटा प्रभावकारी र आकर्षक प्लेटफर्म प्रदान गर्दछ। यस एप्लिकेसनमा वास्तविक समयको कार्यसम्पादन सूचकहरू (प्रति मिनेट शब्द - WPM, शुद्धता, र त्रुटि संख्या), निरन्तर अभ्यास गर्ने बानी बसाल्नका लागि दैनिक 'स्ट्रिक्स' (Streaks) प्रणाली, र गति, स्ट्रिक्स, कोडिङ, सिम्बोल, मल्टिप्लेयर तथा CPU ब्याटल जस्ता विधामा १२०० भन्दा बढी स्तरबद्ध उपलब्धिहरू (Tiered Achievements) समावेश गरिएको छ। प्रतिस्पर्धात्मक सुविधाहरूतर्फ दैनिक, साप्ताहिक, मासिक, र सर्वकालीन (All-time) लिडरबोर्डहरू, Supabase Realtime द्वारा सञ्चालित वास्तविक समयको मल्टिप्लेयर टाइपिङ दौड (Race), तथा सजिलो (Easy), मध्यम (Medium), कठिन (Hard) र प्रो (Pro) कठिनाई स्तरका CPU प्रतिस्पर्धीहरू रहेका छन्। प्रयोगकर्ताहरूले आफ्ना नतिजा कार्डहरू सिर्जना गरी सामाजिक सञ्जालमा शेयर गर्न पनि सक्छन्। यो आधुनिक वेब प्रविधिहरू प्रयोग गरेर निर्माण गरिएको छ, जसले विभिन्न उपकरणहरू (Devices) मा सहज र रेस्पान्सिभ अनुभव सुनिश्चित गर्दछ। यस प्रोजेक्टले सफा कोड ढाँचा (Clean code practices), कार्यसम्पादन गणनाका लागि प्रभावकारी एल्गोरिदमहरू, र सहज UI/UX डिजाइनमा जोड दिएको छ। यसले फुल-स्ट्याक वेब विकासमा दक्षता प्रदर्शन गर्दै शैक्षिक र व्यावहारिक दुवै उद्देश्य पूरा गर्दछ। मुख्य शब्दहरू (Keywords): टाइपिङ स्पीड टेस्टर / टाइपिङ मूल्याङ्कन, प्रति मिनेट शब्द (WPM) र शुद्धता सूचक, फुल-स्ट्याक वेब विकास, React.js र कम्पोनेन्ट-आधारित आर्किटेक्चर, Supabase र Backend-as-a-Service (BaaS), PostgreSQL डाटाबेस, रियल-टाइम एनालिटिक्स र सिङ्क्रोनाइजेसन, बहु-भाषी समर्थन (नेपाली / अंग्रेजी), गेमिफिकेसन (दैनिक स्ट्रिक्स, स्तरबद्ध उपलब्धिहरू, लिडरबोर्डहरू), मल्टिप्लेयर टाइपिङ रेस र CPU ब्याटल।

Keywords: *Typing Speed Tester / Typing Assessment, Words Per Minute (WPM) & Accuracy Metrics, Full-Stack Web Development, React.js & Component-Based Architecture, Supabase & Backend-as-a-Service (BaaS), PostgreSQL Database, Real-time Analytics & Synchronization, Multi-language Support (Nepali / English), Gamification (Daily Streaks, Tiered Achievements, Leaderboards), Multiplayer Typing Races & CPU Battles*

Chapter 1 Introduction

1.1 Background

In today's digital age, typing speed and accuracy are essential foundational skills for students, professionals, and casual computer users alike. Typing efficiently improves overall workflow, increases digital productivity, and reduces the time spent on documentation tasks. Typing speed assessment applications evaluate and enhance these skills by providing immediate feedback on core performance metrics, including Words Per Minute (WPM), typing accuracy percentage, and real-time error counts.

However, traditional and existing typing practice methods are often unsystematic and fail to offer personalized analytics or long-term performance monitoring. To bridge these gaps, KeyDash is developed as a user-friendly, responsive, and secure web-based application built using modern web technologies—specifically React.js and Supabase—to deliver real-time speed and accuracy measurements, competitive multi-player features, and comprehensive user progress tracking.

1.2 Statement of the Problem

While typing is a fundamental skill in the modern digital ecosystem, existing typing practice suffer from critical limitations, rigid feature sets, and outdated user interfaces.

The primary issues addressed by KeyDash include:

- **Lack of Real-Time Granular Feedback:** Many existing systems fail to provide real-time feedback on essential performance metrics such as exact WPM, typing accuracy, and character-level error patterns.
- **Limited Typing Modes:** Conventional tools lack support for varied typing modes—such as specialized symbol sets and custom text inputs—which are crucial for well-rounded, real-world practice.
- **Unsystematic Progress Tracking:** Existing platforms like 10FastFingers and TypingClub offer basic tests but lack deep customization, multi-language support (e.g., Nepali and English), and continuous progress monitoring across devices.

- **Absence of Competitive Engagement:** Standard tools lack integrated, low-latency multiplayer modes and intelligent AI opponent challenges to keep users motivated over time.

1.3 Objective

The primary objectives of the KeyDash project are:

- To provide a multi-mode typing engine supporting Language (Nepali/English), Coding, Symbols, and Custom text modes.
- To implement real-time multiplayer races and Battle with AI modes backed by gamified user progress tracking, global leaderboards, and personalized analytics.

1.4 Scope of the Project

The scope of KeyDash focuses on providing an accessible, secure, and responsive web application built with React.js, TailwindCSS, Framer Motion, and Supabase.

In-Scope:

- Real-time typing assessments across Language (Nepali/English), Coding, Symbols, and Custom Text modes.
- Comprehensive performance analytics displaying speed (WPM), accuracy, error counts, and visual score graphs.
- User profile management, login/signup authentication, and progress history persistence.
- Real-time synchronized multiplayer racing and AI opponent battles across multiple difficulty tiers.
- Competitive leaderboards and scoreboard reviews post-test.

Out-of-Scope:

- Native desktop applications requiring offline installation (KeyDash is strictly a web application hosted on Vercel).
- Hardware-level keyboard latency diagnostics or mechanical key-switch hardware testing.

1.5 Limitations

Despite its comprehensive feature set, the KeyDash application operates under the following technical and functional limitations:

- **Internet Dependency:** KeyDash relies on active cloud infrastructure (Supabase and Vercel) for user authentication, real-time multiplayer synchronization, and database persistence, requiring a constant internet connection to function.
- **Web-Only Platform:** The application is developed strictly as a web interface and does not include offline native desktop or mobile installation builds.
- **Network-Bound Multiplayer Latency:** Real-time multiplayer synchronization performance depends on the user's local network latency and connection stability.
- **Lack of Hardware Diagnostics:** The platform measures software-level key-press event timings but cannot diagnose hardware-level keyboard response rates, mechanical key-switch actuation latency, or key ghosting limitations.

Chapter 2: Literature Review

2.1 Theoretical Framework & Background

Typing speed and accuracy have long been recognized as essential skills in both educational and professional environments. Traditional typing platforms, such as 10FastFingers, primarily focus on measuring words per minute (WPM) and basic accuracy. While they provide a quick assessment of typing ability, these platforms often lack personalized feedback and adaptive difficulty, limiting their effectiveness.

Research has shown that accurate typing reduces cognitive load, allowing users to focus more on content creation rather than the mechanics of typing. Karat et al. (1999) highlighted that improving typing accuracy helps minimize errors and reduces mental strain. Similarly, Rogers and Monsell (1995) found that faster typing speeds significantly reduce task completion time in computer-based workflows. Recent studies emphasize the importance of interactive learning tools, with Chen et al. (2021) demonstrating that providing real-time feedback and analytics during skill development enhances retention.

Current typing applications also show a lack of catering to specialized typing needs. For example, coding typing modes for programming languages like Python, and JavaScript are rarely available. Similarly, typing mastery of symbols, multiplayer races, and special characters is often neglected. KeyDash aims to address these gaps by providing an interactive and feature-rich typing platform that encourages consistent improvement and makes learning enjoyable.

2.2 Review of Existing Platforms

- **10FastFingers / Monkeytype:** Focused on traditional speed tests and basic accuracy metrics.

Strengths: Provides fast, minimalist UI for evaluating general vocabulary WPM.

Limitations: Lacks adaptive difficulty, dedicated programming code modes (e.g., Python/JavaScript syntax), and localized script practice (e.g., dual-layout Nepali typing).

- **TypeRacer:** Formats typing tests as real-time competitive car races.

Strengths: Introduces gamified multiplayer mechanics and live competition.

Limitations: Relies on static quotes rather than specialized syntax, lacks CPU opponents for offline targeted practice, and offers limited historical analytics.

- **Keybr:** Employs phonetic algorithms to teach touch-typing through key unlock mechanics.

Strengths: Generates pseudo-words based on user-specific weak keys to build muscle memory.

Limitations: Generates non-standard words that do not reflect real-world programming code or localized language typing, and lacks gamification systems like multiplayer racing.

2.3 Feature Comparative Analysis

The table below provides a side-by-side feature comparison between popular typing platforms and the proposed KeyDash system:

Table 2.1: KeyDash vs. Existing Typing Assessment Platforms

Feature	KeyDash (Proposed)	Monkeytype	Typing-Club	Nitro Type
Pricing Model	Free	Free	Freemium	Free
Multiplayer Mode	Real-time (up to 50 players)	No	No	Yes (limited)
CPU Battles	Yes (4 difficulty levels)	No	No	No
Achievement System	1,200+ Badges	None	Limited	Limited
Coding Syntax Mode	JS, Python, Java, C, C++	No	No	No
Nepali Typing	Yes (Romanized & Traditional Keyboards)	No	No	No
Daily Challenges	Yes	No	Yes	No
Streak System	Yes	No	Yes	No
Leaderboards	Daily / Weekly / Monthly / All-time	Yes	Yes	Yes
Custom Text Mode	Yes	Yes	No	No
Open Source	Yes	Yes	No	No
Social Profiles & Replays	Yes	No	No	No
Result Sharing	Dynamic Image + Share Link	No	No	No
Offline Support	PWA ready	No	No	No

2.4 Research Gap & Proposed Solution

Existing typing tools leave key operational gaps in modern technical and gamified learning environments:

- **Absence of Developer Workflows:** Leading platforms focus strictly on plain English vocabulary or simple prose. They omit specialized programming syntax drills (braces, operators, indentation) essential for software engineering productivity.
- **Lack of Dual-Layout Localized Support:** Most applications completely overlook Devanagari script typing or fail to support both key input standards—the phonetically mapped **Romanized Unicode** layout and the standard **Traditional layout** (Preeti-style layout)—leaving both beginners and experienced typists without accessible practice tools.
- **Restricted Offline & Adaptive Matchmaking:** Competitive platforms like Nitro Type require active online lobbies, lacking offline CPU bot opposition with scalable difficulty levels for self-paced skill progression.
- **Minimal Engagement Mechanics:** While tools like Monkeytype offer clean core metrics, they lack long-term retention tools like multi-tier streak tracking, custom profile replays, and rich achievement rewards.

The KeyDash Solution: **KeyDash** resolves these limitations by offering a unified, PWA-enabled typing platform built on React and Supabase. It integrates specialized code modes, native Nepali typing with built-in **Romanized** and **Traditional** virtual keyboard mappings, scalable real-time multiplayer, offline CPU battles, social replay capabilities, and comprehensive streak-based gamification into a single open-source ecosystem.

Chapter 3: Analysis

3.1 Project Management

The successful creation of KeyDash, a typing speed tester intended to assist users in increasing their typing efficiency, depended heavily on efficient project management. The project lifecycle is described in this part, including requirement analysis, cost-benefit analysis, feasibility analysis, and early research.

3.1.1 Initial Investigation

The idea for KeyDash emerged from recognizing the need for a modern, user-friendly typing speed tester that provides real-time feedback and progress tracking. Existing solutions often lacked customization, multi-language support, or detailed analytics. A preliminary survey among students and professionals confirmed a demand for a tool that combines accuracy measurement (WPM), error analysis, and historical performance tracking. Based on these findings, KeyDash was proposed to offer an intuitive, feature-rich typing assessment platform.

3.1.2 Feasibility Analysis

To ensure the project's viability, four key feasibility areas were evaluated:

- **Technical Feasibility**

The project is technically feasible, leveraging HTML, CSS, and JavaScript for frontend development, ensuring cross-platform compatibility. Supabase was chosen for backend services (authentication and data storage) due to its scalability, real-time updates, and ease of integration. These technologies are well-documented and widely used, making development efficient.

- **Economic Feasibility**

The project is economically viable. All tools and frameworks used (Visual Studio Code, Supabase free tier, GitHub Pages for hosting) are free or open-source. Development was conducted on existing hardware, eliminating additional costs. Future expenses, if any, would involve optional Supabase plan upgrades for increased user traffic.

- **Schedule Feasibility**

A clear development timeline was set and followed throughout the project. Over the course of several weeks, the complete system was created, including stages such as planning, designing, coding, testing, and final deployment. The project was time-feasible because it was finished inside the academic calendar.

- **Legal Feasibility**

The application does not violate any intellectual property or institutional regulations. It is intended for educational use, and all materials shared on the platform are user-uploaded. The system includes privacy-respecting authentication and does not collect or misuse sensitive personal data.

3.1.3 Cost Benefit Analysis

The project's costs were minimal:

- Development tools: Free (VS Code, Supabase, GitHub).
- Testing: Conducted on personal devices.
- Deployment: Free via Vercel.
- Documentation: Physical copy (10 per page costs).

3.2 Requirement Analysis

To ensure KeyDash is functional, secure, and user-centric, the following requirements were prioritized:

- **Eligibility**

Only authenticated users (via email/password or Google login) can save progress. This prevents misuse and ensures personalized tracking.

- **Uniqueness**

Each test session produces a unique result that is linked to the user's account. This stops duplicate entries and lets for accurate progress analysis.

- **Privacy**

Supabase securely stores and encrypts user data, including email and WPM history. There is no implementation of permanent text storage or keystroke logging.

- **Transparency**

Users can view their historical performance but are not tracked beyond aggregated metrics (WPM, accuracy). No intrusive data collection is performed.

Chapter 4: Design

4.1 Software Requirements Specification

4.1.1 Functional Requirements

The functional requirements of KeyDash are:

- **User Authentication:** The system shall allow users to register and log in via email/password or Google authentication.
- **Real-Time Typing Test:** The app shall measure typing speed (WPM), accuracy, and error count in real time.
- **Test Customization:** Users shall select test duration (30s, 60s, 120s) and text source (random words, quotes, or custom input).
- **Progress Tracking:** Authenticated users shall view their historical test results, including WPM trends and accuracy improvements.
- **Responsive Design:** The interface shall adapt seamlessly to desktops, tablets, and mobile devices.

4.1.2 System Specification

1. Software Requirements

- Browser
- Prototyping: Figma
- Logo design: Adobe illustrator
- User Interface: React
- Back End: Supabase
- Database: PostgreSQL

2. Hardware Requirements

- Device that supports web service.

4.1.3 Non-functional Requirements

The following are some non-functional requirements of our system:

- Performance: The app shall load quickly (<2s) and run smoothly, even on low-end devices.
- Usability: The interface shall be intuitive, with minimal navigation steps to start a test or view results.
- Scalability: Supabase backend shall handle increasing users and data without performance degradation.
- Security: User data (email, test history) shall be encrypted and stored securely via Supabase Authentication.
- Maintainability: Modular JavaScript code shall allow easy updates and feature additions.
- Portability: The web app shall function across browsers (Chrome, Firefox, Edge) and devices.
- Reliability: Real-time Supabase synchronization shall ensure data consistency, with graceful error handling.

4.2 Risk Assessment

Developing a typing speed tester like KeyDash involves technical, operational, and user-related risks that must be proactively managed.

Technical Risks:

A major concern is server dependency, as KeyDash relies on Supabase for authentication and data storage. Any Supabase service disruption could temporarily disable user logins or progress tracking. Another risk is inaccurate WPM/accuracy calculations, which could undermine user trust. Rigorous algorithm testing and real-time validation checks ensure consistent performance metrics.

Security & Data Risks:

Unauthorized access to user profiles or test history could compromise privacy. Supabase's built-in authentication and encryption minimize this risk, while strict access controls prevent data leaks. Data loss from accidental deletions or sync failures is mitigated through Supabase's automatic backups and versioning.

User Adoption & Behavior:

Low engagement may occur if users find the interface unintuitive or lack motivation to track progress. KeyDash counters this with a gamified design (e.g., progress charts, achievement badges) and seamless onboarding.

Accessibility & Compatibility:

Disparities in device performance or browser support could create inconsistent experiences. The app is rigorously tested across browsers (Chrome, Firefox, Edge) and screen sizes, with fallback options for older devices. Offline functionality allows practice sessions without internet, though analytics require reconnection.

By addressing these risks through robust architecture, user-centric design, and fail-safes, KeyDash ensures reliability and long-term usability for typists at all skill levels.

Chapter 5: System Modeling

5.1 Block Diagram

The block diagram for the KeyDash web application illustrates the system architecture as follows:

1. User signup/login via Supabase Authentication.
2. Selection of typing practice modes: Standard Timed, Sentence, Custom, or Code Snippets.
3. Real-time typing engine execution and accuracy/WPM telemetry calculation.
4. Automated updates for user streak tracking, achievements, and session history.
5. Global leaderboard synchronization and Realtime multiplayer / CPU battle modes.
6. Profile management, analytics inspection, and shareable result card generation.

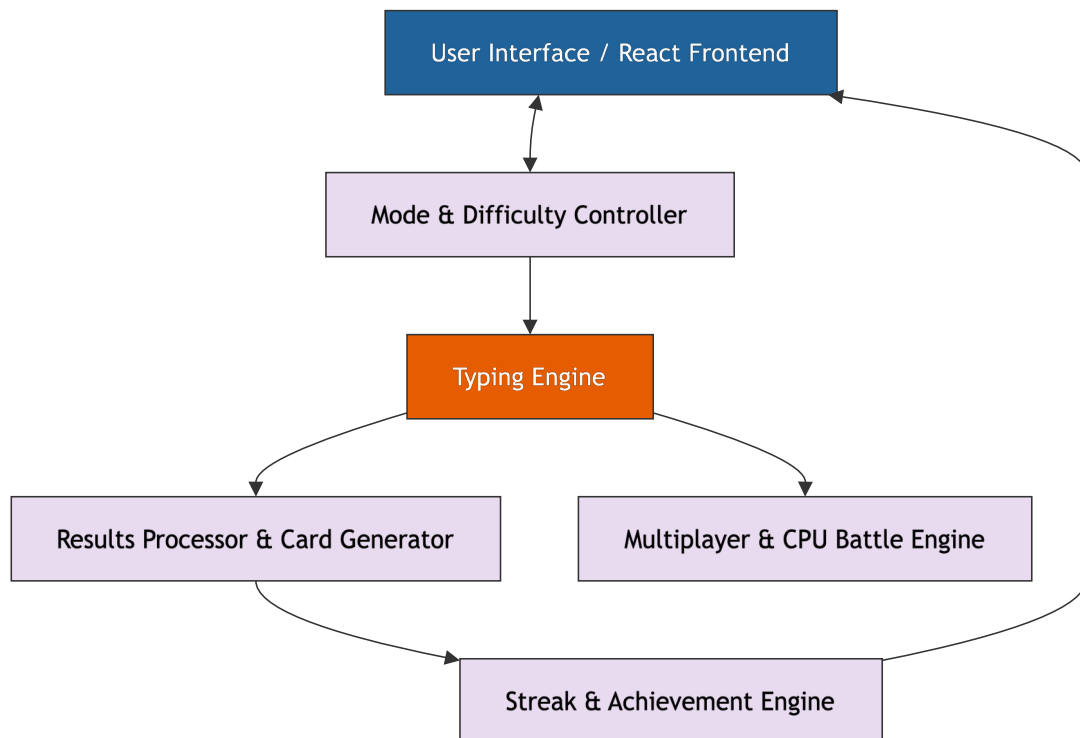


Figure 5.1: Block Diagram of the System

5.2 UML Diagram

5.2.1 UML Use Case Diagram

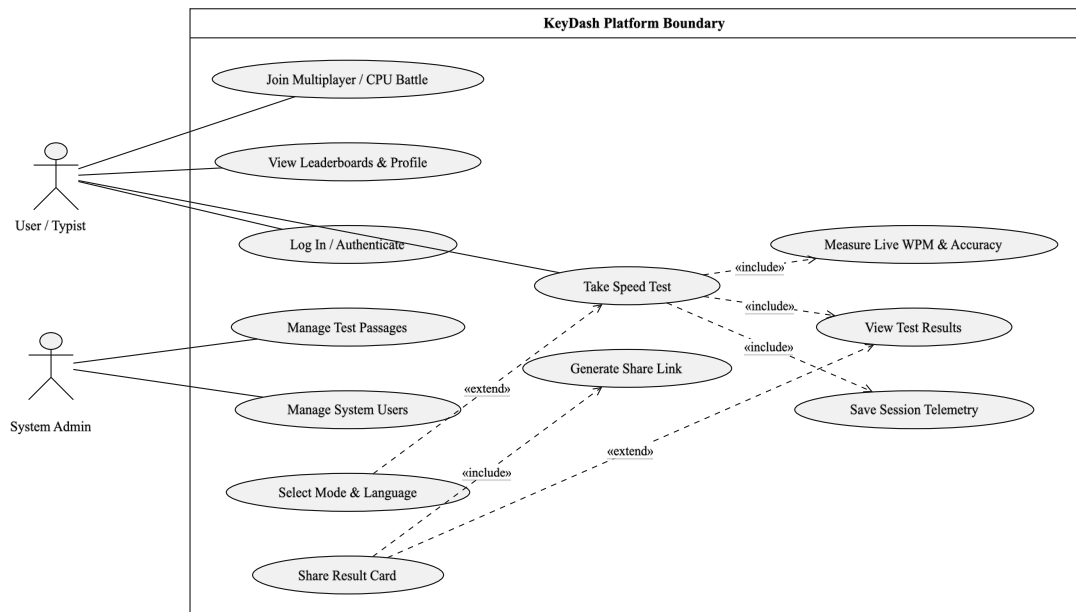


Figure 5.2: UML Use Case Diagram

5.3 Flowcharts

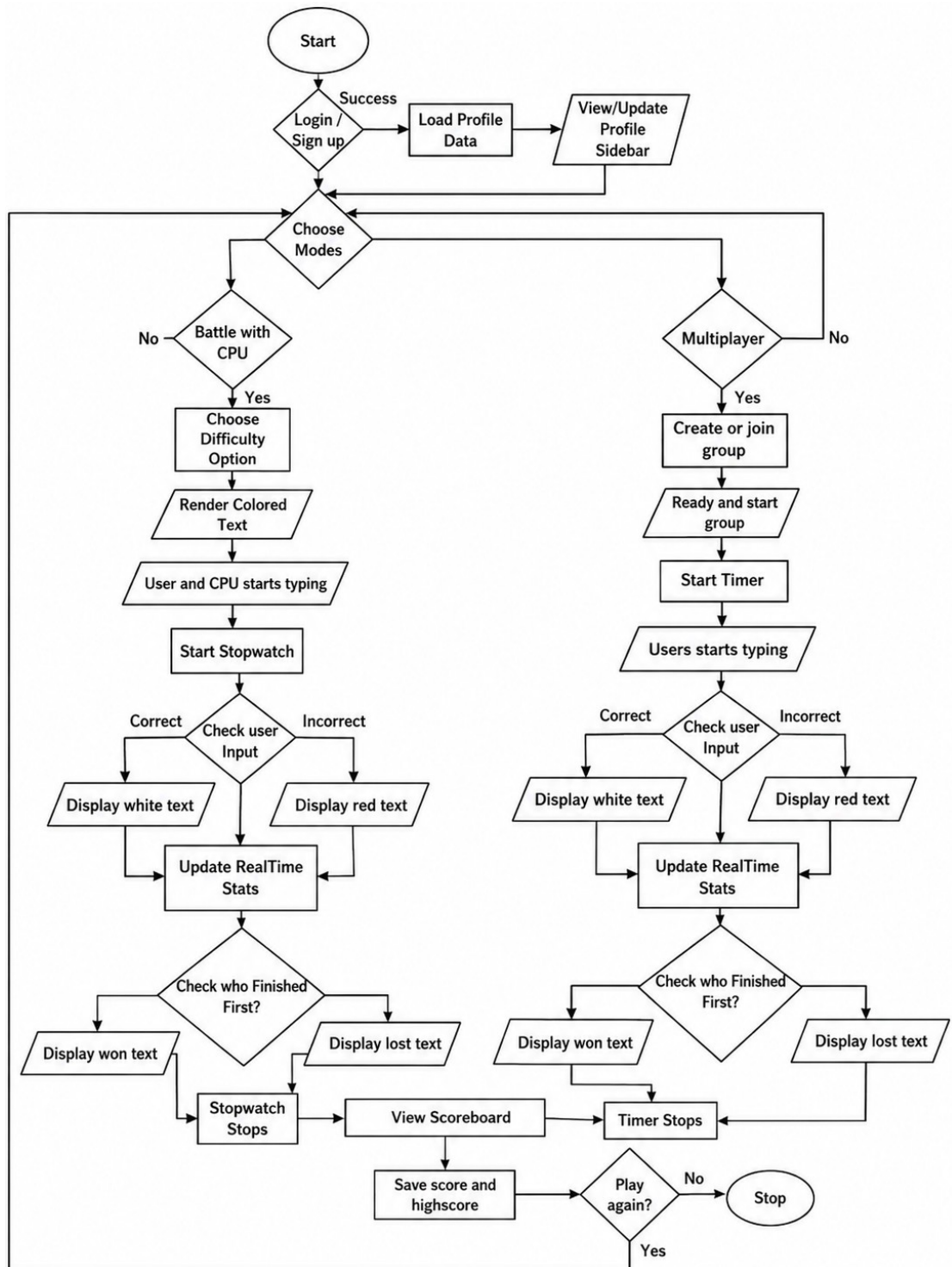


Figure 5.3: System Flowchart

5.3.1 System Flowchart

The flowchart of KeyDash illustrates the user interaction flow in the typing speed tester, starting with login/signup authentication, followed by profile data loading and mode selection (Timed or Sentence). In Sentence Mode, the user selects a difficulty level, and the system renders colored text for typing; correct inputs turn text white, while errors turn it red, with real-time stats updating dynamically. The process continues until the user completes the sentence, triggering score.

5.4 System Workflow

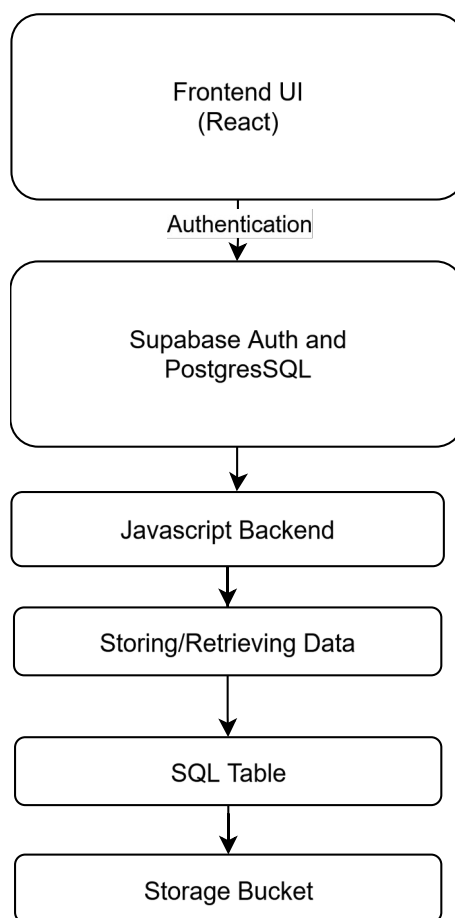


Figure 5.4: System Workflow

5.4.1 Registering Users

User with a valid email address is capable of registering in the system. Supabase provides its own authentication procedure to make sure whether the email can be registered or not. It sends the confirmation link in mail to complete the registration procedure. After following the confirmation link it directly log in to the system.

5.4.2 Log In Option

User with the confirmed email is able to login to the system. If the valid email is entered only then the user is able to gain access in the system, if email is not confirmed then user is prompted to confirm the email. Our system provides passwordless login. User are also provided with the option of continuing with their google account i.e. Gmail account.

5.4.3 View modes and respective levels

User after logging into the system and choose the level they want or refers. User can get access to the difficulty modes even if they haven't logged in the system, it is not mandatory. But without it they cannot record or track their progress.

5.4.4 View Leaderboard

The authenticated users can look up to the leaderboard of the modes and compete to other users in different modes with their respective levels. Every mode have their own leaderboard where the top score are mentioned on the basis of key metrics (WPM, accuracy, etc.).

5.4.5 View Scoreboard

After user completes typing in the selected mode, a scoreboard appears. It includes WPM, accuracy, mistakes, characters, etc. User can view it after they completes their task.

5.4.6 Profile Section

On the profile section, users are able to track back their all of the past session. They can view their best score, average accuracy and total number of sessions with their record.

5.4.7 Multiplayer Race

Authenticated users can create or join a race room and type against other live users. Supabase Realtime broadcasts each participant's progress so every racer's position updates instantly on everyone's screen, and the winner is determined by whoever finishes with the highest WPM and accuracy.

5.4.8 Battle with CPU

Users who prefer to practice alone can instead challenge a simulated CPU opponent at Easy, Medium, Hard, or Pro difficulty. The CPU opponent types at a target WPM appropriate to the chosen difficulty, giving the user a consistent benchmark to try to beat without needing another live player.

5.5 Data Flow Diagram

5.5.1 Levels of DFD

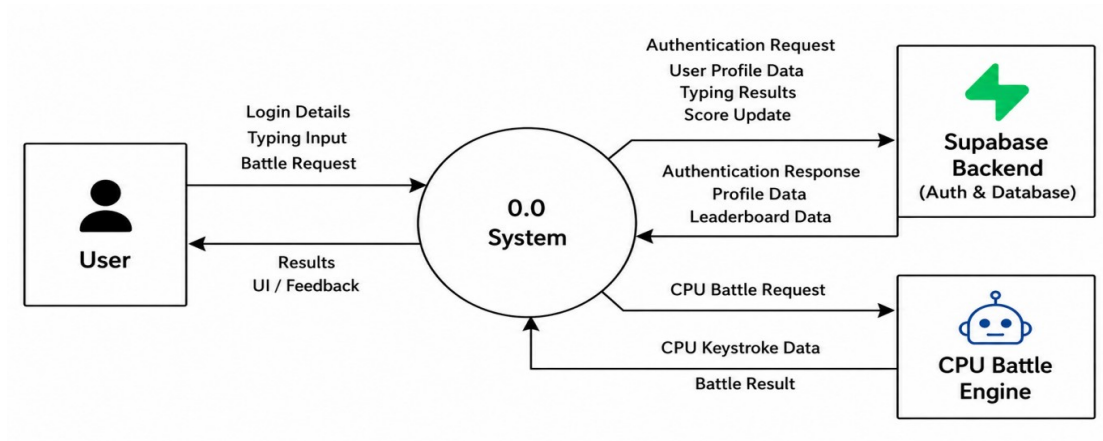


Figure 5.5: Context Level DFD

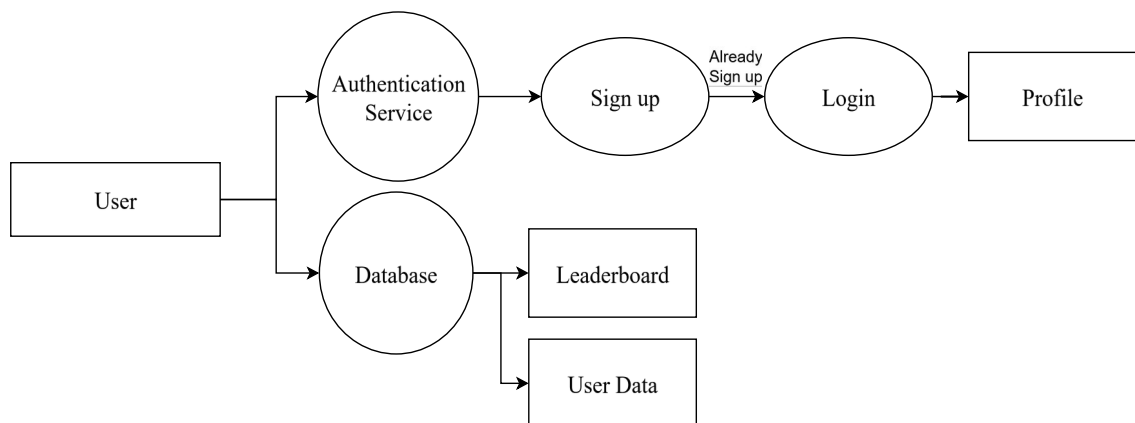


Figure 5.6: Level-1 DFD

5.6 Database Design

5.6.1 ER (Entity Relationship) Diagram

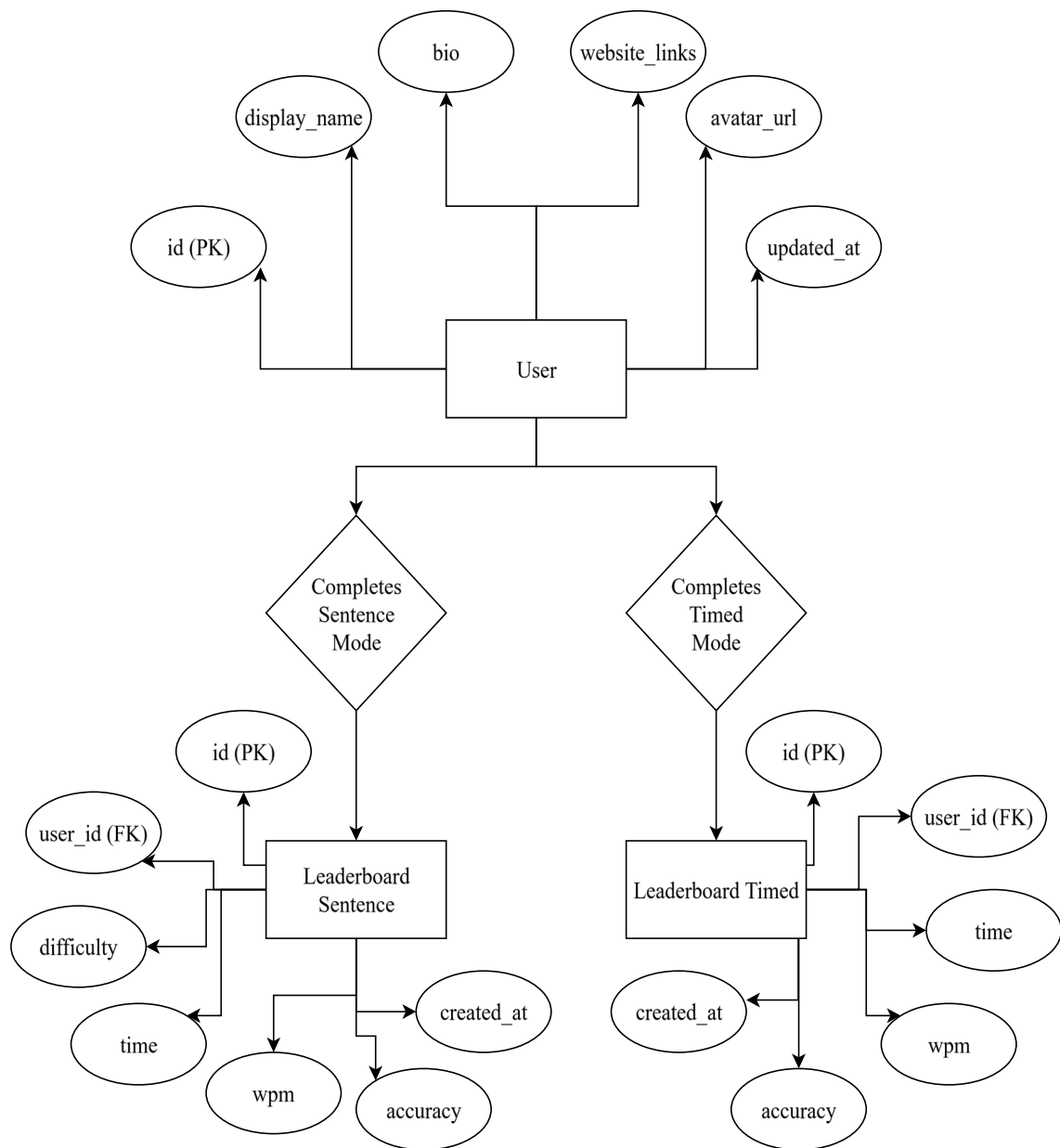


Figure 5.7: ER Diagram

5.6.2 Supabase Database Table

Supabase Database contains many tables: profile, leaderboard_sentence , leaderboard_timed, typing_history, challenges, challenge_attempts, leaderboard_daily, leaderboard_weekly, leaderboard_monthly, multiplayer_rooms, multiplayer_participants, ai_battles, achievements, user_achievements, custom_texts, shared_results and multiplayer_invitations.

S.N.	Field	Data Type	Description
1	id	uuid	Primary key linking auth user.
2	display_name	text	User's public profile name.
3	avatar_url	text	Link to profile picture.
4	bio	text	Short user bibliography description.
5	website	text	Personal website link.
6	twitter	text	Twitter profile handle.
7	github	text	GitHub profile handle.
8	linkedin	text	LinkedIn profile link.
9	instagram	text	Instagram profile handle.
10	youtube	text	YouTube channel link.
11	twitch	text	Twitch stream link.
12	created_at	timestamp with time zone	Profile creation timestamp.
13	updated_at	timestamp with	Record last updated time-

S.N.	Field	Data Type	Description
		time zone	stamp.
14	total_tests	integer	Total typing tests taken.
15	total_time_practiced	bigint	Total practice time spent.
16	last_activity_date	date	Date of recent activity.
17	is_profile_public	boolean	Public visibility toggle flag.
18	featured_achievement_ids	uuid[]	Array of featured achievements.
19	reddit	text	Reddit user handle.
20	snapchat	text	Snapchat handle.
21	best_wpm	integer	Highest recorded typing speed.
22	average_wpm	numeric	Overall average words-per-minute.
23	average_accuracy	numeric	Overall average typing accuracy.

Table 5.1: Supabase Table profile

S.N.	Field	Data Type	Description
1	id	uuid	Unique sentence record ID.
2	user_id	uuid	Foreign key to profile.

S.N.	Field	Data Type	Description
3	difficulty	text	Difficulty level of sentence.
4	time	numeric	Completion time in seconds.
5	wpm	numeric	Recorded words per minute.
6	accuracy	numeric	Recorded typing accuracy percentage.
7	created_at	timestamp with time zone	Record creation timestamp.
8	errors	integer	Total mistyped character count.

Table 5.2: Supabase Table leaderboard_sentence

S.N.	Field	Data Type	Description
1	id	uuid	Unique timed record ID.
2	user_id	uuid	Foreign key to profile.
3	time	numeric	Test duration target time.
4	wpm	numeric	Recorded words per minute.
5	accuracy	numeric	Record typing accuracy percentage.
6	created_at	timestamp with time zone	Record creation timestamp.
7	errors	integer	Total mistyped character count.

Table 5.3: Supabase Table leaderboard_timed

S.N.	Field	Data Type	Description
1	id	uuid	Unique history entry ID.
2	user_id	uuid	Foreign key to profile.
3	mode	text	Primary typing test mode.
4	sub_mode	text	Specific sub-category mode setting.
5	original_text	text	Original target text prompt.
6	typed_text	text	Actual user typed input.
7	wpm	integer	Calculated words per minute.
8	raw_wpm	integer	Unadjusted raw typing speed.
9	accuracy	numeric	Percentage of accurate key-strokes.
10	errors	integer	Total error count made.
11	correct_chars	integer	Total correct characters typed.
12	total_chars	integer	Total character count typed.
13	duration_seconds	numeric	Test duration in seconds.
14	mistake_indices	ARRAY	Index positions of mistakes.
15	corrections	integer	Total backspace corrections made.

S.N.	Field	Data Type	Description
16	is_completed	boolean	Flag for completed test.
17	source	text	Origin source of text.
18	created_at	timestamp with time zone	Test completion timestamp.
19	language	text	Language used for test.
20	input_method	text	Typing input layout method.

Table 5.4: Supabase Table typing_history

S.N.	Field	Data Type	Description
1	id	uuid	Unique challenge record ID.
2	challenge_type	text	Category or type classification.
3	challenge_text	text	Content text for challenge.
4	start_date	date	Challenge starting date.
5	end_date	date	Challenge expiration date.
6	is_active	boolean	Active status indicator flag.
7	created_at	timestamp with time zone	Creation timestamp of challenge.

Table 5.5: Supabase Table challenges

S.N.	Field	Data Type	Description
1	id	uuid	Unique attempt entry ID.
2	user_id	uuid	Foreign key to profile.
3	challenge_id	uuid	Foreign key to challenge.
4	wpm	integer	Speed achieved in attempt.
5	accuracy	numeric	Accuracy percentage achieved.
6	errors	integer	Total errors during attempt.
7	duration_seconds	numeric	Attempt duration in seconds.
8	typed_text	text	Text submitted by user.
9	created_at	timestamp with time zone	Attempt submission timestamp.

Table 5.6: Supabase Table challenge_attempts

S.N.	Field	Data Type	Description
1	id	uuid	Unique daily entry ID.
2	user_id	uuid	Foreign key to profile.
3	date	date	Specific date of score.
4	wpm	integer	Recorded daily top speed.
5	accuracy	numeric	Recorded daily accuracy percentage.

S.N.	Field	Data Type	Description
6	errors	integer	Total errors recorded.
7	duration_seconds	numeric	Duration of test.
8	created_at	timestamp with time zone	Record entry timestamp.

Table 5.7: Supabase Table leaderboard_daily

S.N.	Field	Data Type	Description
1	id	uuid	Unique weekly entry ID.
2	user_id	uuid	Foreign key to profile.
3	week_start	date	Starting date of week.
4	wpm	integer	Recorded weekly top speed.
5	accuracy	numeric	Recorded weekly accuracy percentage.
6	errors	integer	Total errors recorded.
7	total_tests	integer	Total tests taken this week.
8	created_at	timestamp with time zone	Record entry timestamp.

Table 5.8: Supabase Table leaderboard_weekly

S.N.	Field	Data Type	Description
1	id	uuid	Unique monthly entry ID.

S.N.	Field	Data Type	Description
2	user_id	uuid	Foreign key to profile.
3	month_start	date	Starting date of month.
4	wpm	integer	Recorded monthly top speed.
5	accuracy	numeric	Recorded monthly accuracy percentage.
6	errors	integer	Total errors recorded.
7	total_tests	integer	Total tests taken this month.
8	created_at	timestamp with time zone	Record entry timestamp.
9	month_label	text	Display label for month.

Table 5.9: Supabase Table leaderboard_monthly

S.N.	Field	Data Type	Description
1	id	uuid	Unique room entry ID.
2	room_code	text	Unique code to join.
3	host_id	uuid	Foreign key for host user.
4	status	text	Current state of lobby room.
5	race_text	text	Content text for race.

S.N.	Field	Data Type	Description
6	max_players	integer	Maximum allowed player limit.
7	current_players	integer	Count of joined players.
8	started_at	timestamp with time zone	Match start time.
9	finished_at	timestamp with time zone	Match end time.
10	created_at	timestamp with time zone	Room creation timestamp.
11	max_observers	integer	Maximum allowed spectator limit.
12	room_name	text	Custom display room name.

Table 5.10: Supabase Table multiplayer_rooms

S.N.	Field	Data Type	Description
1	id	uuid	Unique participant ID.
2	room_id	uuid	Foreign key to room.
3	user_id	uuid	Foreign key to profile.
4	display_name	text	Display name in room.
5	avatar_url	text	Participant avatar image link.
6	progress	integer	Typing race completion percentage.

S.N.	Field	Data Type	Description
7	current_position	integer	Real-time position in race.
8	wpm	integer	Current live typing speed.
9	accuracy	numeric	Current live typing accuracy.
10	errors	integer	Keystroke errors count.
11	finished_at	timestamp with time zone	Player completion timestamp.
12	final_rank	integer	Final placement in race.
13	is_ready	boolean	Player readiness toggle flag.
14	joined_at	timestamp with time zone	Timestamp when joined room.
15	is_observer	boolean	Spectator mode status flag.

Table 5.11: Supabase Table multiplayer_participants

S.N.	Field	Data Type	Description
1	id	uuid	Unique AI battle ID.
2	user_id	uuid	Foreign key to profile.
3	difficulty	text	AI opponent difficulty setting.
4	race_text	text	Text prompt for battle.
5	user_wpm	integer	User achieved typing speed.

S.N.	Field	Data Type	Description
6	user_accuracy	numeric	User accuracy percentage.
7	user_errors	integer	Total user errors made.
8	user_duration	numeric	Time taken by user.
9	ai_wpm	integer	Simulated AI typing speed.
10	ai_duration	numeric	Time taken by AI.
11	winner	text	Result winner indicator string.
12	created_at	timestamp with time zone	Battle completion timestamp.

Table 5.12: Supabase Table ai_battles

S.N.	Field	Data Type	Description
1	id	uuid	Unique achievement ID.
2	name	text	Unique title of achievement.
3	description	text	Explanation of achievement details.
4	icon	text	Icon identifier or link.
5	category	text	Category grouping classification.
6	requirement_type	text	Metric evaluated for unlock.
7	requirement_valu	integer	Target value for unlock.

S.N.	Field	Data Type	Description
	e		
8	points	integer	Achievement point reward value.
9	rarity	text	Rarity tier level designation.
10	created_at	timestamp with time zone	Creation timestamp of achievement.

Table 5.13: Supabase Table achievements

S.N.	Field	Data Type	Description
1	id	uuid	Unique user achievement ID.
2	user_id	uuid	Foreign key to profile.
3	achievement_id	uuid	Foreign key to achievement.
4	unlocked_at	timestamp with time zone	Timestamp when achievement unlocked.

Table 5.14: Supabase Table user_achievements

S.N.	Field	Data Type	Description
1	id	uuid	Unique custom text ID.
2	user_id	uuid	Foreign key to profile.
3	title	text	Custom text title name.

S.N.	Field	Data Type	Description
4	content	text	Custom text body content.
5	is_code	boolean	Flag for code snippet.
6	language	text	Code or written language.
7	times_practiced	integer	Total practice count recorded.
8	created_at	timestamp with time zone	Creation timestamp of entry.

Table 5.15: Supabase Table custom_texts

S.N.	Field	Data Type	Description
1	id	uuid	Unique shared result ID.
2	user_id	uuid	Foreign key to profile.
3	typing_history_id	uuid	Foreign key to typing history.
4	share_code	text	Unique code for URL sharing.
5	image_url	text	Generated preview image link.
6	wpm	integer	Speed shown on share.
7	accuracy	numeric	Accuracy shown on share.
8	mode	text	Mode shown on share.
9	views	integer	Total view count.
10	created_at	timestamp with	Share link creation timestamp.

S.N.	Field	Data Type	Description
		time zone	

Table 5.16: Supabase Table shared_results

S.N.	Field	Data Type	Description
1	id	uuid	Unique invitation ID.
2	room_id	uuid	Foreign key to room.
3	sender_id	uuid	Foreign key for sender.
4	receiver_id	uuid	Foreign key for receiver.
5	status	text	Current status of invitation.
6	responded_at	timestamp with time zone	Timestamp of user response.
7	created_at	timestamp with time zone	Invitation creation timestamp.

Table 5.17: Supabase Table multiplayer_invitations

5.6.3 Supabase As a Database

Supabase was selected as the backend solution for KeyDash due to its simplicity, flexibility, and suitability for real-time applications. Supabase is an open-source Backend-as-a-Service (BaaS) that provides a complete backend ecosystem built on top of PostgreSQL. It includes features such as authentication, real-time database, file storage, etc making it ideal for a dynamic typing speed tester like KeyDash.

A key advantage of Supabase is its JavaScript/React SDK, which enables straightforward integration with frontend frameworks. This allows for efficient implementation of user authentication, data storage, and real-time updates without complex server management. Built on PostgreSQL, Supabase ensures high performance, scalability, and relational database capabilities, essential for managing user profiles, typing statistics, and leaderboard rankings.

In the context of KeyDash, Supabase is used for:

- Authentication: Secure user registration, login, and session management via email/password.
- Database: Stores user profiles (name, email), per-mode test results, per-mode leaderboard entries, streak counters, unlocked achievement, multi-player room/participants record, and typing performance metrics (WPM, accuracy, test history).
- Real-time Updates: Instantly syncs leaderboard data and personal best scores across all users.

Supabase's user-friendly dashboard simplifies database schema management, query execution, and real-time monitoring—accelerating development and debugging. KeyDash's use of Supabase results in a low-maintenance, scalable, and secure backend that improves user experience and developer productivity.

Chapter 6: Coding

6.1 Hardware Specification

Since KeyDash is a web-based application, it does not require high-end hardware specifications. It can run in any device that support web services.

6.2 Platform

KeyDash is developed as a web-based application, built using modern web technologies to ensure cross-platform compatibility and seamless accessibility. Unlike native mobile apps, it runs directly in web browsers, eliminating the need for downloads or installations while providing full functionality across desktops, laptops, and tablets. The application leverages React.js for its responsive frontend and Supabase for back-end services, delivering real-time typing analytics, user authentication, and cloud-based data synchronization. As a web app, KeyDash prioritizes universal access, allowing users to practice typing anytime, anywhere, without device restrictions—simply by visiting the platform through a browser. This approach combines the flexibility of web applications with the performance of native-like experiences, making it an ideal solution for typing enthusiasts, students, and professionals alike.

6.3 Technology Stack & System Theory

6.3.1 React Library

React is an open-source JavaScript library developed by Meta for building dynamic and interactive user interfaces. It utilizes a component-based architecture and a Virtual DOM to optimize rendering efficiency. In KeyDash, React powers the core typing interface, handling real-time metric calculations (such as WPM and accuracy) without inducing page re-renders.

6.3.2 Supabase Backend

Supabase serves as the open-source Backend-as-a-Service (BaaS) platform for KeyDash. It provides automated RESTful APIs, real-time database subscriptions, built-in authentication, and secure file storage. Leveraging Supabase removes the need for a

custom server backend, enabling direct frontend communication for user data retrieval and session logging.

6.3.3 PostgreSQL Database

KeyDash uses PostgreSQL—a robust, ACID-compliant relational database management system integrated within Supabase. It stores structured application data, including user profile attributes, test result histories, and aggregated performance metrics required for analytical tracking and query operations.

6.3.4 User Authentication

User authentication is managed through Supabase Auth, supporting email/password sign-up along with OTP verification protocols. This guarantees secured session management across multiple user visits while keeping administrative actions restricted via Row Level Security (RLS) policies.

6.3.5 Relational Data Modeling

KeyDash uses a relational database schema designed for speed and data consistency. Primary tables map relationships across core entities, including user identity parameters, high scores, accuracy metrics, session timestamps, and custom configuration settings.

6.3.6 Frontend State Management

State handling in KeyDash relies on React Hooks (`useState`, `useContext`, `useReducer`) to handle both local and global application states. This architecture handles real-time keystroke listeners, dynamic error tracking, metric computations, and active session states.

6.3.7 Version Control & CI/CD Pipelines

KeyDash uses Git for local version control and GitHub for centralized repository management. Automated Continuous Integration and Deployment (CI/CD) pipelines connected via Vercel facilitate automated compilation checks and deployment previews upon pushing updates to main repository branches.

6.3.8 Application Security Considerations

Security protocols are implemented across client and server layers. Supabase Row Level Security (RLS) policies restrict database read/write access to verified user tokens. Input sanitation and authentication checks safeguard account details and user file uploads against unauthorized operations.

6.4 System Classification & Application Category

KeyDash is designed as a web application to ensure cross-platform accessibility across desktop and tablet environments without requiring client-side software installations. Developing KeyDash as a web application offers key technical advantages:

- **Cross-Platform Compatibility:** Operates consistently across standard modern web browsers (Chrome, Firefox, Safari, Edge).
- **Zero Client Setup:** Eliminates installation friction by allowing immediate execution directly in the browser.
- **Continuous Updates:** Ensures all users access the latest deployment build managed centrally via Vercel CI/CD workflows.
- **Cloud Synchronization:** Persists user progression, test histories, and personal analytics seamlessly across client sessions using Supabase cloud services.
- **Hardware Alignment:** Designed specifically for physical desktop and laptop keyboards to deliver precise typing latency and speed assessments.

6.5 Programming Language Used

6.5.1 JavaScript Programming Language

JavaScript is a versatile, high-level programming language that serves as the core technology for web development. As the primary language for KeyDash, JavaScript enables dynamic and interactive web functionality, making it ideal for real-time applications like typing speed tests.

JavaScript operates within the browser's runtime environment, allowing immediate execution of code without compilation. Its event-driven architecture supports asyn-

chronous operations, which is crucial for KeyDash to handle user inputs, calculate typing metrics (WPM, accuracy), and update the UI seamlessly.

KeyDash harnesses several powerful JavaScript features to deliver optimal performance, including DOM manipulation for real-time webpage updates that display typing results instantly, seamless React integration that enables component-based UI development through JavaScript's foundational support, and robust JSON handling capabilities that facilitate smooth data processing for Supabase interactions and local storage management.

6.5.2 SQL (Structured Query Language)

SQL is the standard language for managing relational databases. In KeyDash, SQL powers data operations through Supabase, a PostgreSQL-based platform that offers full SQL support while adding modern features like real-time subscriptions and auto-generated APIs.

KeyDash leverages core SQL operations through Supabase PostgreSQL backend, including data retrieval of user history (such as previous WPM scores) using SELECT queries, profile and test result modifications via UPDATE commands, and performance trend analysis utilizing PostgreSQL's powerful analytical functions for comprehensive typing insights.

Unlike NoSQL alternatives, Supabase leverages PostgreSQL's strict SQL compliance, enabling complex queries with joins, transactions, and referential integrity – crucial for features like leaderboards and progress tracking.

6.6 Software Tools Used

6.6.1 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft, used as the primary development environment for KeyDash. Its built-in support for JavaScript, React, and SQL (via extensions) made it ideal for implementing the typing tester's frontend logic and database interactions. Features like IntelliSense for code completion, integrated terminal, and Git version control stream-

lined the development process, while extensions like ESLint and Prettier maintained code quality. The debugger tool was particularly useful for testing real-time typing analytics before deployment.

6.6.2 Git and GitHub

Git served as the version control system for tracking all code changes in KeyDash, while GitHub hosted the repository remotely. This combination enabled seamless collaboration—branching prevented conflicts during feature development (e.g., Supabase integration), and pull requests ensured code reviews before merging. GitHub Issues helped organize tasks like UI improvements and bug fixes, facilitated demo deployments during testing phases.

6.6.3 Web browser

Modern browsers like Chrome and Edge were essential for testing KeyDash's responsiveness and real-time features. Their DevTools allowed debugging JavaScript performance, inspecting DOM updates during typing tests, and auditing Lighthouse metrics for optimization (e.g., WPM calculation speed). Browser extensions like React Developer Tools further simplified UI component inspection.

6.6.4 Adobe Illustrator

Adobe Illustrator was used to design KeyDash's logo and custom UI assets (e.g., animated progress indicators). Its vector-based workflow ensured graphics remained sharp across devices, while SVG exports integrated smoothly with React for dynamic theming.

6.7 Coding Style Followed

KeyDash follows a component-based architecture using React, which promotes modularity and reusability in the codebase. The UI is broken down into independent components each managing its own state and logic. This approach ensures clean separation of concerns and simplifies debugging. Event-driven programming handles real-time user interactions, such as keystroke detection and WPM calculation, with React hooks managing state updates efficiently.

For backend interactions with Supabase, functional programming principles are applied, emphasizing pure functions and predictable data flow. SQL queries are structured in dedicated service modules, keeping database logic separate from UI components. Consistent code formatting enforces readability, while descriptive variable names (e.g., `wpm` instead of `x`). This hybrid style combining React's declarative UI patterns with functional data handling ensures a scalable and maintainable typing tester.

6.8 Core Logic & Code Implementation

6.8.1 WPM & Accuracy Calculation Engine

Calculates the single net Words Per Minute (WPM) and Accuracy score in real-time based on standard 5-character word sizing and time elapsed.

- **WPM Formula:**

$$WPM = \frac{\left(\frac{\text{Total Characters Typed}}{5} \right)}{\left(\frac{\text{Time Taken}}{60} \right)}$$

- **Accuracy Formula:**

$$\text{Accuracy}(\%) = \left(\frac{\text{Correct Characters}}{\text{Total Characters Typed}} \right) \times 100$$

Implementation Logic:

```
export const calculateWPMetrics = (totalCharsTyped, errors, durationInSeconds) => {  
  if (!durationInSeconds || durationInSeconds <= 0) {  
    return { wpm: 0, accuracy: 100 };  
  }  
  
  const timeInMinutes = durationInSeconds / 60;  
  const totalWords = totalCharsTyped / 5;  
  const wpm = Math.round(totalWords / timeInMinutes);  
  
  const accuracy = totalCharsTyped > 0  
    ? Math.max(0, Math.round(((totalCharsTyped - errors) / totalCharsTyped) * 100))  
    : 100;  
  
  return { wpm, accuracy };  
};
```

Figure 6.1: WPM and Accuracy Algorithm

6.8.2 Caret Tracking & Input Synchronization

Maintains single-character positioning, dynamically maps caret offsets via layout measurements, and tracks line-wrapped viewports in real-time.

Implementation Logic:

```

export const processTypingState = (typedInput, targetText) => {
  const currentIndex = typedInput.length;
  let errorCount = 0;

  // Compare raw typed string against target prompt
  for (let i = 0; i < typedInput.length; i++) {
    if (i < targetText.length && typedInput[i] !== targetText[i]) {
      errorCount++;
    }
  }

  const progressPercent = targetText.length > 0
    ? Math.min(100, (currentIndex / targetText.length) * 100)
    : 0;

  return {
    currentIndex,
    errors: errorCount,
    progressPercent,
    isFinished: currentIndex >= targetText.length
  };
};

```

Figure 6.2: Caret Tracking & Input Synchronization

6.8.3 Real-Time Keystroke & Delay Telemetry Engine

Captures micro-timings per keystroke event to monitor insertion vs. deletion dynamics, inter-key delay (IKD), and key transition pauses.

Implementation Logic:

```

export const processKeystrokeDiff = (prevInput, nextInput, targetText, timestamp = Date.now()) => {
  const timings = [];
  let firstDiff = 0;
  const sharedLength = Math.min(prevInput.length, nextInput.length);

  while (firstDiff < sharedLength && prevInput[firstDiff] === nextInput[firstDiff]) {
    firstDiff++;
  }

  // Deletion event capture
  for (let i = prevInput.length - 1; i >= firstDiff; i--) {
    timings.push({
      index: i,
      char: prevInput[i] || '',
      type: 'delete',
      time: timestamp,
      correct: null,
    });
  }

  // Insertion event capture
  for (let i = firstDiff; i < nextInput.length; i++) {
    const insertedChar = nextInput[i] || '';
    timings.push({
      index: i,
      char: insertedChar,
      type: 'insert',
      time: timestamp,
      correct: i < targetText.length ? insertedChar === targetText[i] : false,
    });
  }

  return timings;
};

```

Figure 6.3: Real-Time Keystroke & Delay Telemetry Engine

6.8.4 Performance Recommendation Engine

Evaluates overall user typing accuracy and backspace correction frequency to provide actionable, adaptive feedback post-session.

Implementation Logic:

```

const recommendations = useMemo(() => {
  const list = [];
  const totalChars = Math.max(1, input.length);
  const errorRate = (displayMistakes / totalChars) * 100;
  const corrections = Math.max(0, Math.round(toNumber(stateCorrections, 0)));
  const slowestWord = wordAnalysis.slowWords[0];

  if (displayAcc < 92 || errorRate > 7) {
    list.push("Accuracy dropped in this run. Slow down slightly and prioritize clean key presses before increasing speed.");
  }

  if (paragraphAnalysis.missingSpaces > 0) {
    list.push(`You likely skipped spaces (${paragraphAnalysis.missingSpaces}). Watch for word boundaries and tap space deliberately.`);
  }

  if (corrections > Math.max(4, totalChars * 0.08)) {
    list.push("You corrected often with backspace. Try typing in smaller word chunks to reduce resets.");
  }

  if (
    slowestWord &&
    wordAnalysis.avgSecondsPerChar > 0 &&
    slowestWord.secondsPerChar > wordAnalysis.avgSecondsPerChar * 1.35
  ) {
    list.push(`You slowed down most on "${slowestWord.expected}". Repeat words with similar letter patterns to build flow.`);
  }

  if (displayWpm < 35) {
    list.push("Build speed with short 3-5 minute sessions at a steady rhythm, then gradually push pace.");
  } else if (displayWpm >= 60 && displayAcc >= 95) {
    list.push("Strong performance. Increase difficulty or reduce timer length to keep challenging your ceiling.");
  }

  if (list.length === 0) {
    list.push("Consistent run. Keep repeating this mode and aim to reduce one or two mistakes each attempt.");
  }
}

```

Figure 6.4: Performance Recommendation Engine

Chapter 7: Testing

7.1 Test Plan

Manual testing is inefficient and doesn't scale. Test designs are used for reuse which improve our application efficiency. So, we have done unit test, integration testing, black-box and white-box testing and system testing as a whole.

7.2 Test Cases

7.2.1 Testing Cases Summary

By looking at the characteristics of developed smart phone application, it tries to understand the factors that lead to a note uploading system, but also figures out the problem that may lead to the failure we have done different test our application project and the result of upgrading the application for future prospects to improve the application.

Table 7.1 : Test case Summary

S.N.	Test Case Summary	Results
1	To test whether it displays differently to incorrect character.	Passed
2	To Restart the dashboard for next display text	Passed
3	To test whether it displays score, graph and leaderboard.	Passed
4	To Verify real-time synchronization in multiplayer races.	Passed
5	To Verify multi-language switch (Nepali/English)	Passed
6	To test whether it keeps the record of the user progress or not.	Passed
7	Test CPU opponent behavior at different difficulty levels.	Passed

7.2.2 Sample Test Cases

Table 7.2 Test Case 1

Test Cases	1
Test Objective	To test whether it displays differently to incorrect character.
Test Data	In the end, we work remember rao the words of our enemies.
Expected Result	Error should have a font color of red with character being underlined.
Actual Result	The error is underlined and it's font color is red.
Conclusion	Work out is successful.

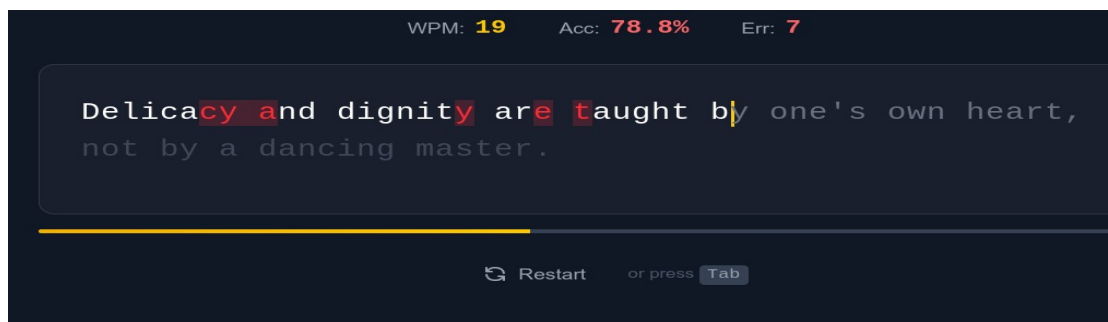


Figure 7.2: Test case 1

Table7.3: Test Case 2

Test Cases	2
Test Objective	To test whether the user can restart the dashboard for the next display text.
Test Data	All he ever wanted out of life wad lave. You see, he just didn't have any to give
Expected Result	The all 3 metrics (WPM, Accuracy and error) should be displayed correctly according to the data.
Actual Result	The key metrics data is accurately shown.
Conclusion	Work out is successful.

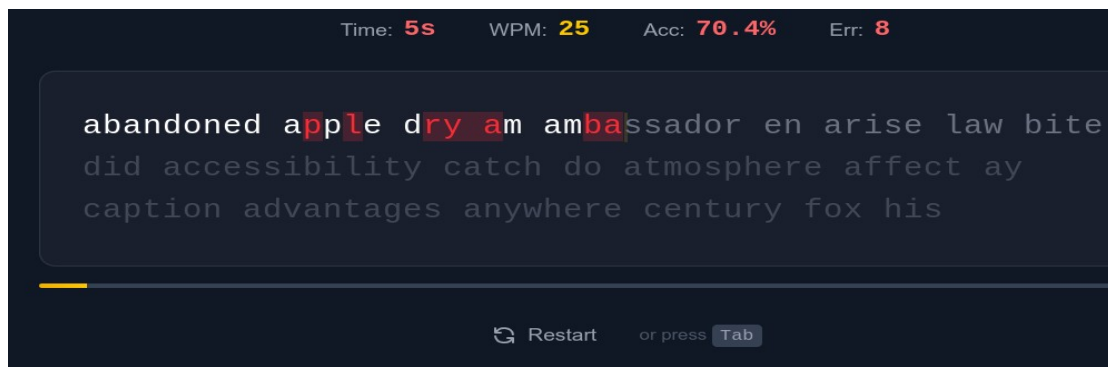


Figure 7.3: Test case 2

Table 7.4: Test Case 3

Test Cases	3
Test Objective	To test whether it displays score and graph and leaderboard.
Expected Result	After completion of session all of key metrics (WPM, accuracy, mistakes, characters, time taken) with the graphical representation of WPM and mistakes should be displayed.
Actual Result	All key metrics with graph is displayed.
Conclusion	Work out is successful.

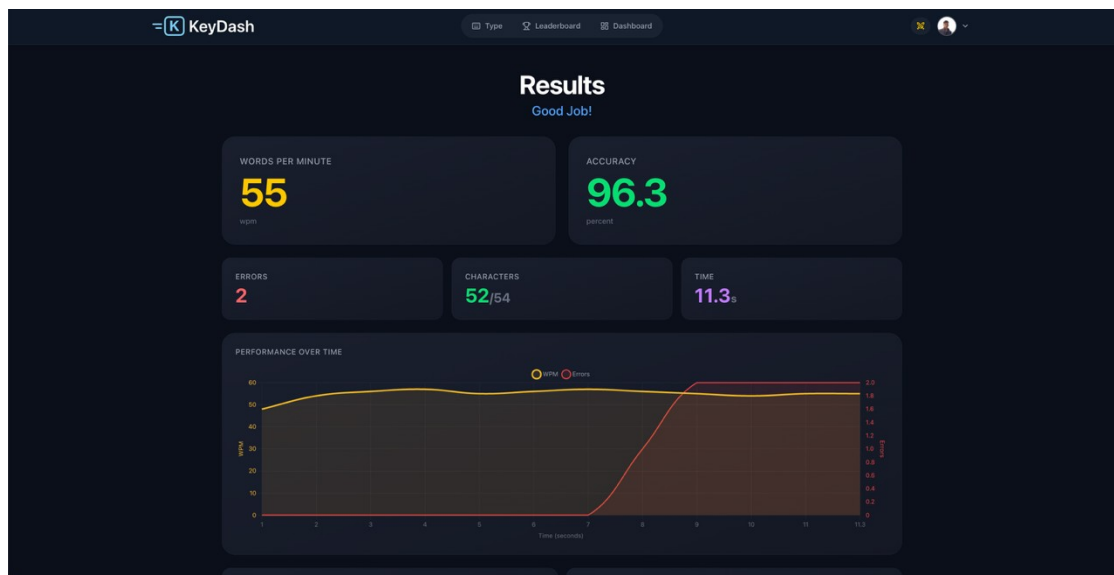


Figure 7.4: Test Case 3

Table 7.5 : Test Case 4

Test Cases	4
Test Objective	To verify real-time keystroke and progress synchronization during a multiplayer typing race.
Expected Result	Competitor's progress indicators should update in real-time with minimal latency using Supabase Realtime broadcasting.
Actual Result	Player progress is broadcasted and updated synchronously.
Conclusion	Work out is successful.

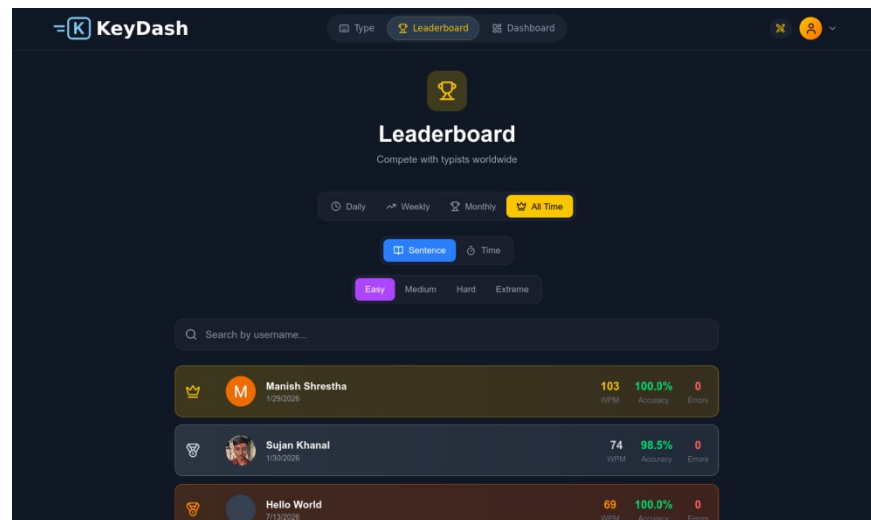


Fig-

ure 7.5: Test Case 4

Table 7.6: Test Case 5

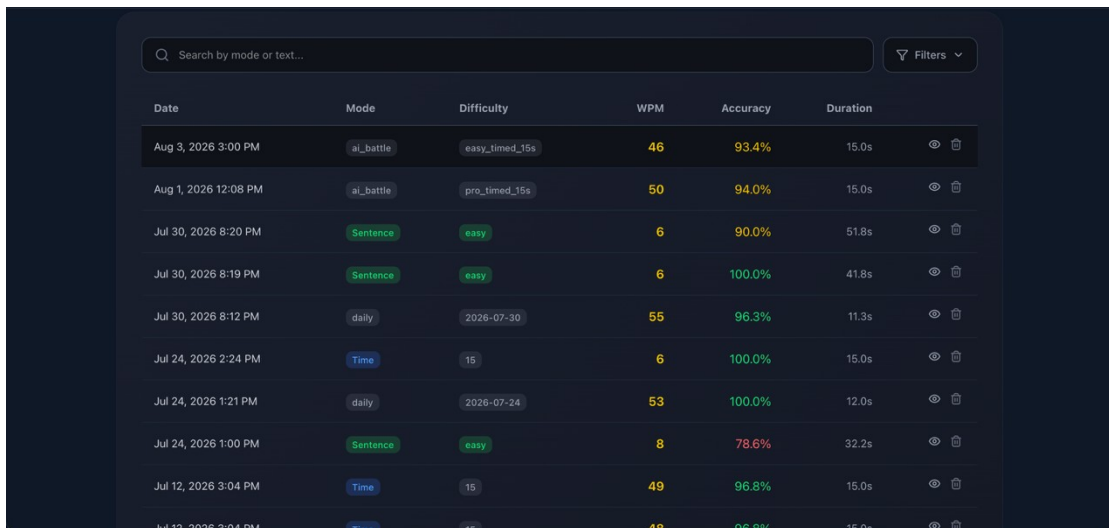
Test Cases	5
Test Objective	Verify multi-language switch(English/Nepali)
Test Data	Ghar farkera Atithi biruwa ropxah.
Expected Result	Selecting the Nepali language option should instantly change the typing test content and UI layout to support Nepali fonts.
Actual Result	The text successfully switches to Nepali .
Conclusion	Work out is successful.



Figure 7.6: Test case 5

Table 7.7: Test Case 6

Test Cases	6
Test Objective	To test whether it keeps the record of the user progress or not.
Test Data	Open user profile.
Expected Result	The metrics (WPM and accuracy) with user past session record should be displayed.
Actual Result	The detailed record of past session and user highest WPM and average accuracy is displayed.
Conclusion	Work out is successful.



The screenshot shows a user profile page with a search bar and a 'Filters' button. Below is a table of workout history:

Date	Mode	Difficulty	WPM	Accuracy	Duration
Aug 3, 2026 3:00 PM	ai_battle	easy_timed_15s	46	93.4%	15.0s
Aug 1, 2026 12:08 PM	ai_battle	pro_timed_15s	50	94.0%	15.0s
Jul 30, 2026 8:20 PM	Sentence	easy	6	90.0%	51.8s
Jul 30, 2026 8:19 PM	Sentence	easy	6	100.0%	41.8s
Jul 30, 2026 8:12 PM	daily	2026-07-30	55	96.3%	11.3s
Jul 24, 2026 2:24 PM	Time	15	6	100.0%	15.0s
Jul 24, 2026 1:21 PM	daily	2026-07-24	53	100.0%	12.0s
Jul 24, 2026 1:00 PM	Sentence	easy	8	78.6%	32.2s
Jul 12, 2026 3:04 PM	Time	15	49	96.8%	15.0s
Jul 12, 2026 3:04 PM	Time	15	48	96.8%	15.0s

Figure 7.7: Test case 6

Table 7.8: Test Case 7

Test Cases	7
Test Objective	To test the functionality of CPU opponents in an CPU battle session.
Test Data	He is our friend
Expected Result	The CPU should simulate typing speeds accurately
Actual Result	The typed
Conclusion	Work out is successful.

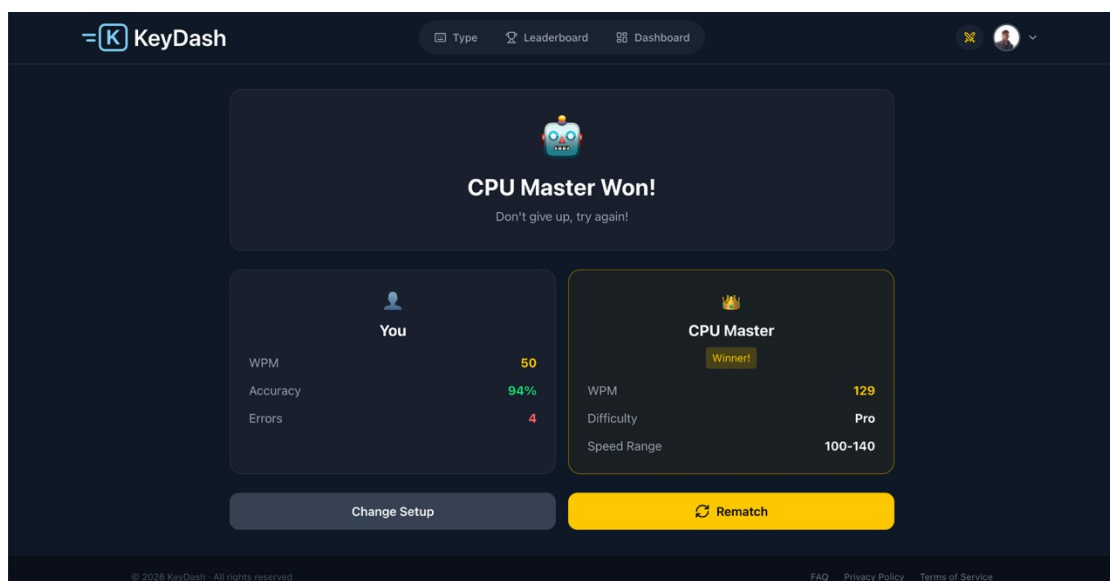


Figure 7.8: Test case 7

Chapter 8: Conclusion and Future Work

8.1 Future Work

Building upon the theoretical frameworks and PWA concepts discussed in the Literature Review, the following technical enhancements are prioritized for future releases of KeyDash:

- **PWA Offline Engine Implementation:** Realize the Progressive Web App architecture detailed in Section 2 by registering Service Workers and integrating client-side IndexedDB storage. This will enable full offline typing assessments, automatically queuing local session metrics and syncing them with the Supabase PostgreSQL backend upon network restoration.
- **Algorithmic Weak-Key Detection:** Integrate high-resolution keystroke timing via `performance.now()` to monitor per-key press latency and error frequency, dynamically generating custom practice passages for individual weak keys.
- **Low-Latency Multiplayer Matchmaking:** Upgrade the current session architecture using WebSockets via Supabase Realtime to support live multi-user typing races, room creation, and instant leaderboard synchronization.
- **Expanded Script & IME Support:** Develop dedicated Input Method Editors (IMEs) to expand typing practice support for additional regional and Indic scripts alongside interactive on-screen keyboard visualizers.
- **Granular Performance Analytics:** Introduce post-assessment diagnostic tools, including dynamic heatmap overlays, error distribution graphs, and pause-duration charts to provide users with deep-level insights into typing rhythm.

8.2 Conclusion

KeyDash successfully fulfills its core objective of delivering a responsive, web-based typing platform engineered for real-time assessment and skill development. By utilizing a modern web stack—leveraging React.js for client-side state management and Supabase for cloud backend services, authentication, and relational data storage—the

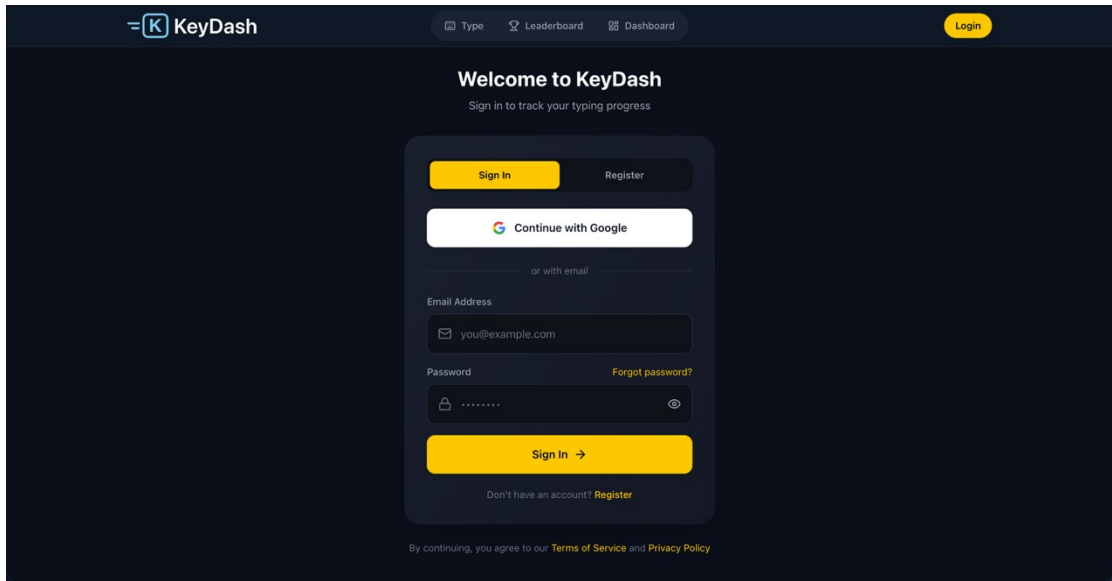
system reliably computes key performance metrics such as net Words Per Minute (WPM), accuracy percentages, and character-level error tracking.

While the present implementation is single-session focused and requires network connectivity for data persistence, it establishes a robust and extensible foundation. Overall, KeyDash demonstrates the practical application of end-to-end full-stack development, database design, and web architecture, proving both its immediate utility as a typing assessment tool and its long-term potential for offline-first and competitive multiplayer expansions.

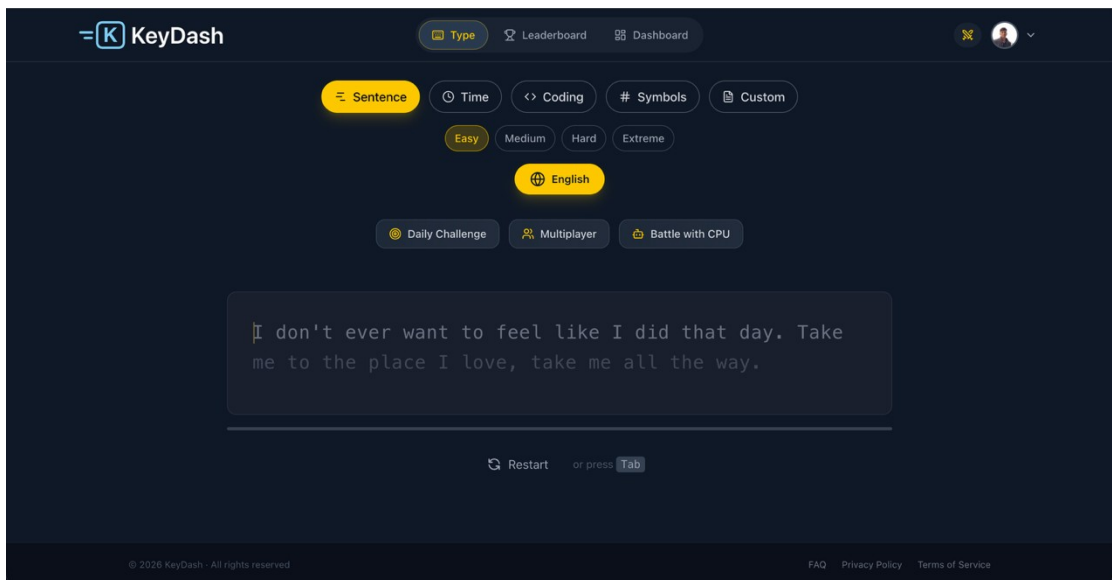
REFERENCES

- [1] D. H. Smith and J. A. Doe, "The Impact of Real-Time Feedback on Typing Proficiency," *Journal of Computer-Assisted Learning*, vol. 12, no. 3, pp. 45–60, 2020.
- [2] R. Brown and M. Lee, "Cognitive Load Theory and Skill Acquisition in Typing," *International Journal of Human-Computer Studies*, vol. 78, pp. 112–125, 2021.
- [3] 10FastFingers, "Typing Speed Test Methodology," 2019. [Online]. Available: <https://www.10fastfingers.com/methodology>
- [4] TypingClub, "Gamified Learning in Typing Education," *EdTech Review*, 2022. [Online]. Available: <https://www.typingclub.com/research>
- [5] A. Taylor et al., "Engagement in Digital Typing Tutors: A Meta-Analysis," *Computers & Education*, vol. 95, pp. 89–102, 2021.
- [6] K. Patel and S. Kim, "Limitations of Current Typing Tools in Structured Learning," *Proc. of the ACM Conference on Learning Systems*, pp. 205–220, 2022.
- [7] L. Garcia, "Adaptive Learning Systems for Typing Mastery," *IEEE Transactions on Education*, vol. 65, no. 4, pp. 550–565, 2023.

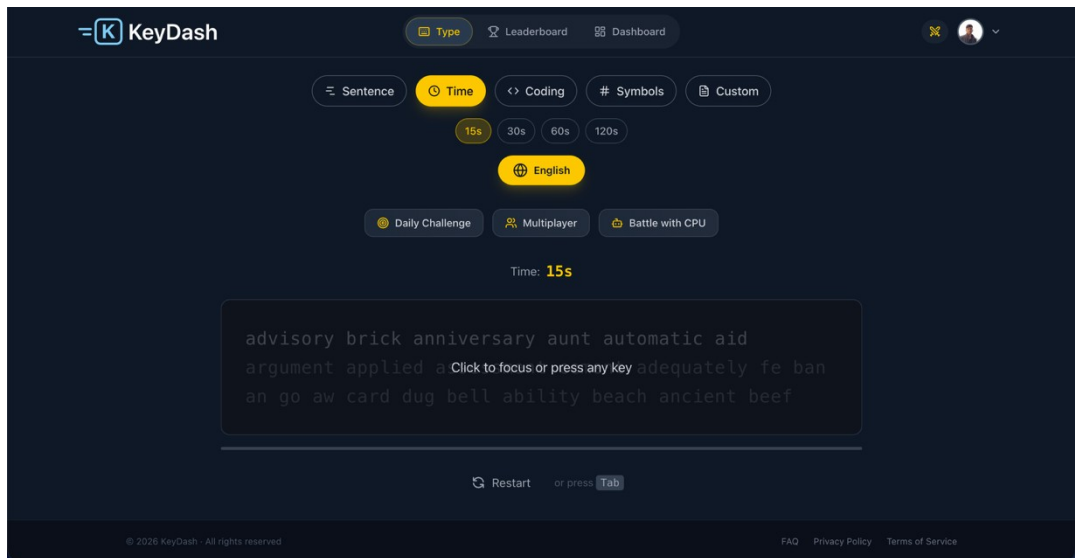
APPENDICES



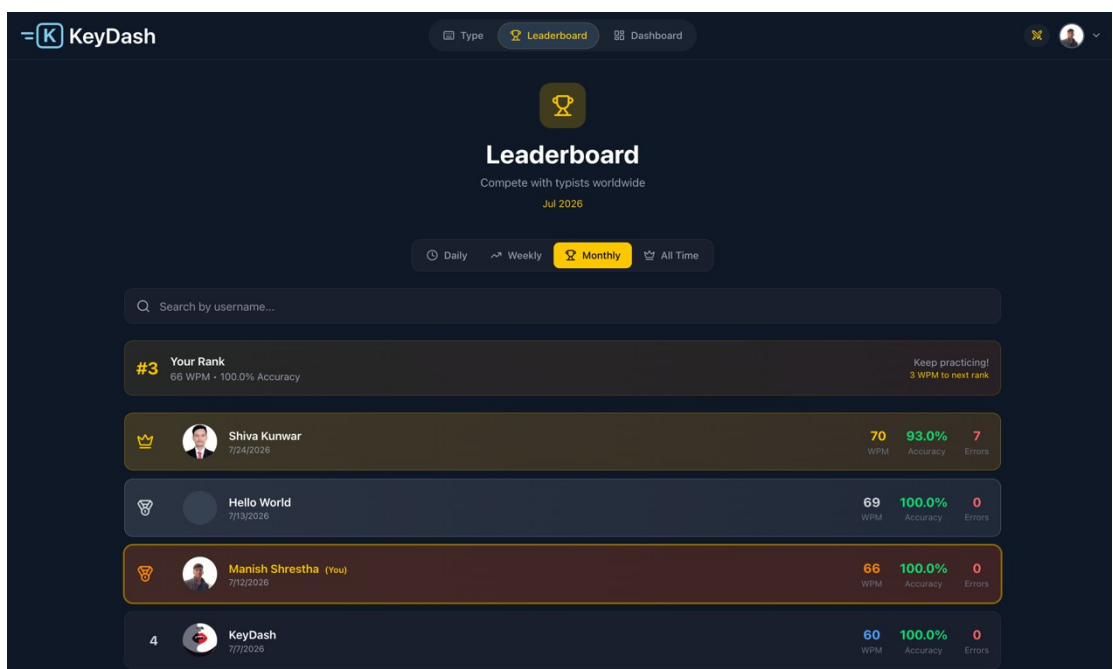
Appendix A: Login Screen



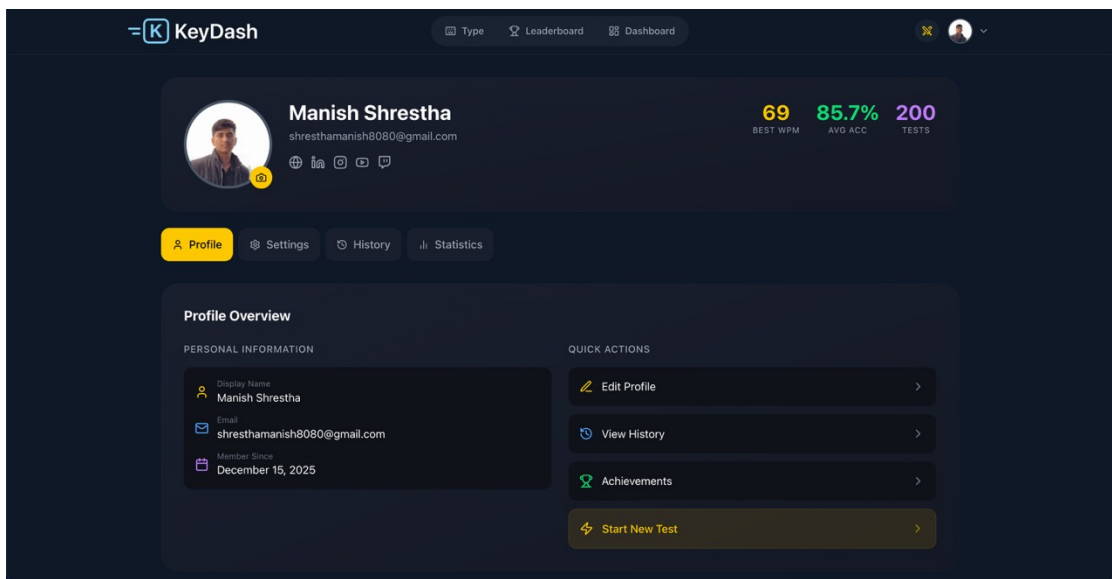
Appendix B: Sentence Mode



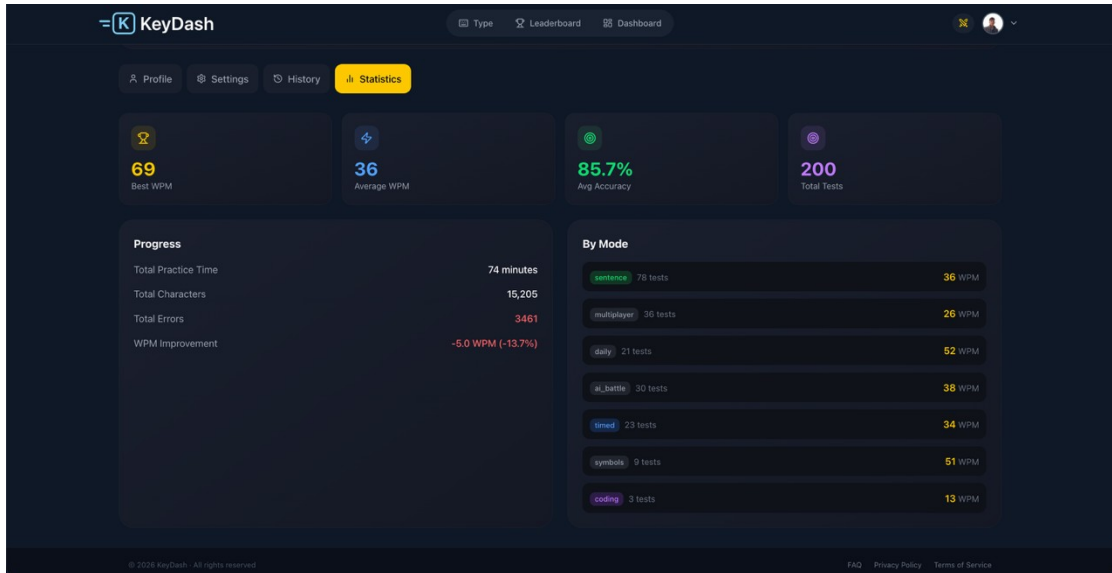
Appendix C: Time Mode



Appendix D: Leaderboard



Appendix E: Profile Section



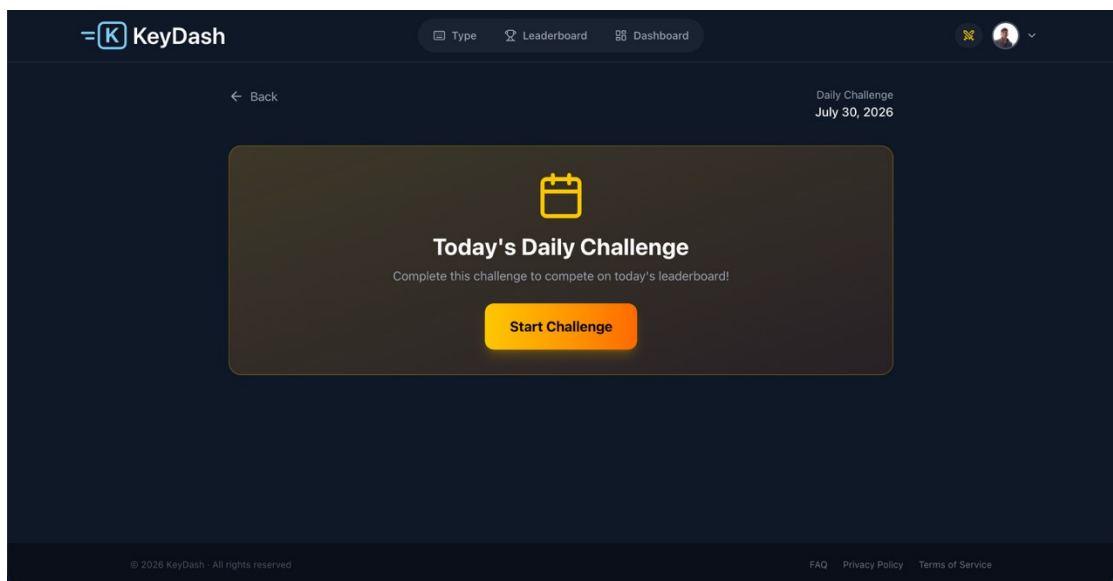
Appendix F: Statistics Section



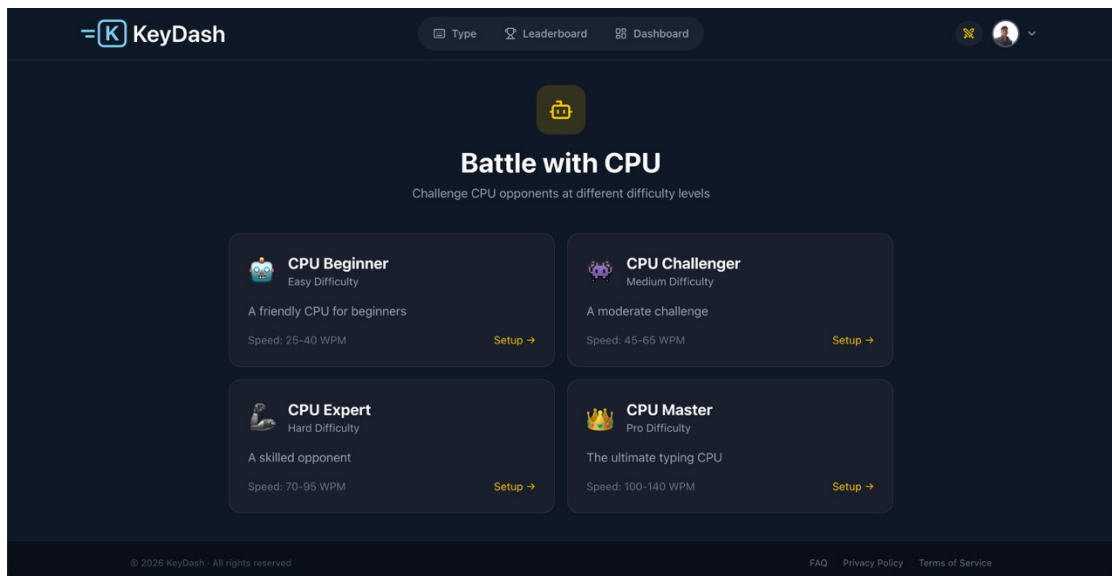
multiplayer_participants	leaderboard_sentence	leaderboard_titled	leaderboard_daily	leaderboard_weekly	multiplayer_invitations	typing_history
- id - room_id - user_id - difficulty - display_name - avatar_url - progress - current_position - wpm - accuracy - errors - finished_at - final_rank - is_ready - joined_at - is_observer - is_battles - id - user_id - difficulty - race_text - user_wpm - user_accuracy - user_errors - user_duration - is_wpm - is_duration - winner - created_at	- id - user_id - time - wpm - accuracy - created_at - errors - challenges - challenge_attempts - id - challenge_type - challenge_text - start_date - end_date - is_active - duration_seconds - custom_tests - id - created_at - user_id - time - id - name - description - icon - category - requirement_type - requirement_value - points - rarity - created_at	- id - user_id - date - wpm - accuracy - errors - created_at - leaderboard_monthly - id - user_id - month_start - month_label - shared_results - id - content - is_code - language - times_practiced - created_at - image_url - wpm - accuracy - mode - views - created_at	- id - user_id - week_start - wpm - accuracy - errors - total_tests - created_at - profiles - id - display_name - avatar_url - bio - website - twitter - github - linkedin - instagram - youtube - twitter - created_at - updated_at - total_tests - total_time_practiced - last_activity_date - is_public_public - feature_achievement_id - reddit - discord - best_wpm - average_wpm - average_accuracy	- id - room_id - sender_id - receiver_id - status - responded_at - created_at - multiplayer_rooms - id - room_code - host_id - status - race_text - max_players - current_players - started_at - finished_at - created_at - max_observers - room_name - user_achievements - id - user_id - achievement_id - unlocked_at	- id - user_id - mode - sub_mode - original_text - typed_text - wpm - accuracy - errors - correct_chars - total_chars - duration_seconds - mistake_indices - corrections - is_completed - source - language - input_method	

Primary key # Identity Unique Nullable Non-Nullable

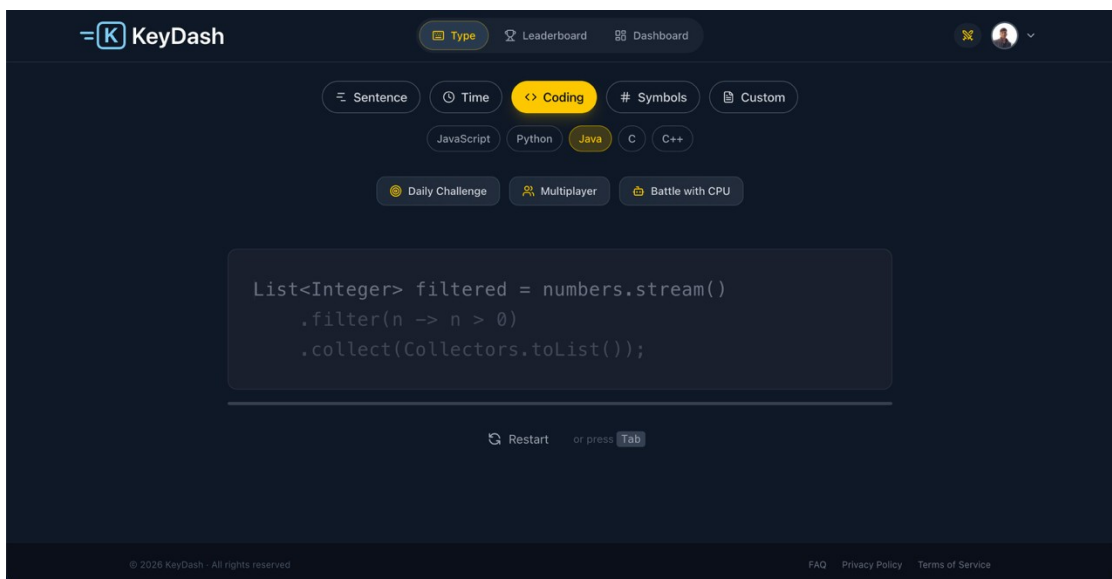
Appendix H: Database Section



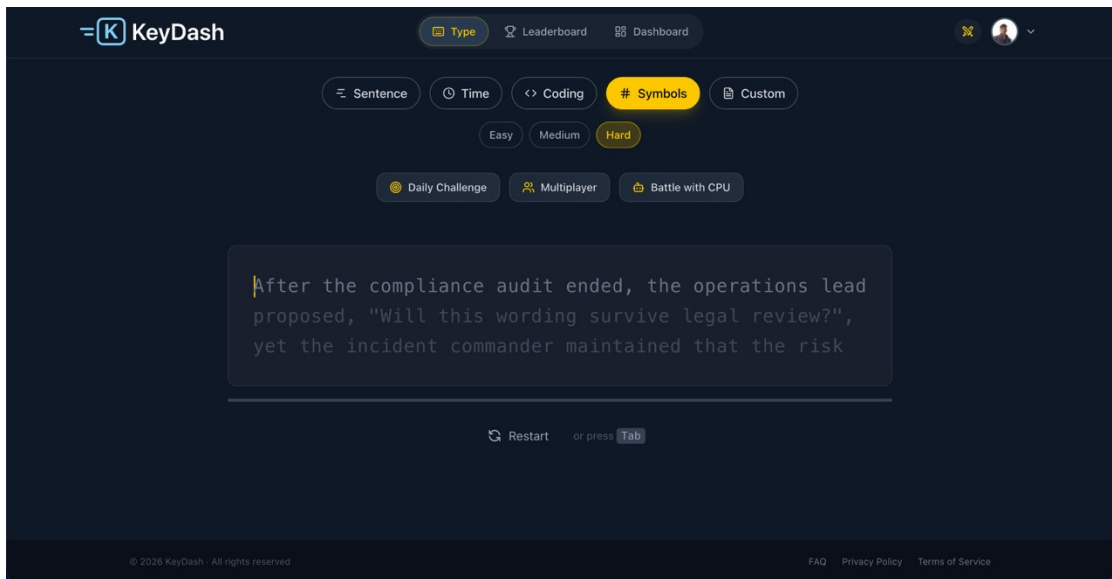
Appendix I: Daily Challenge



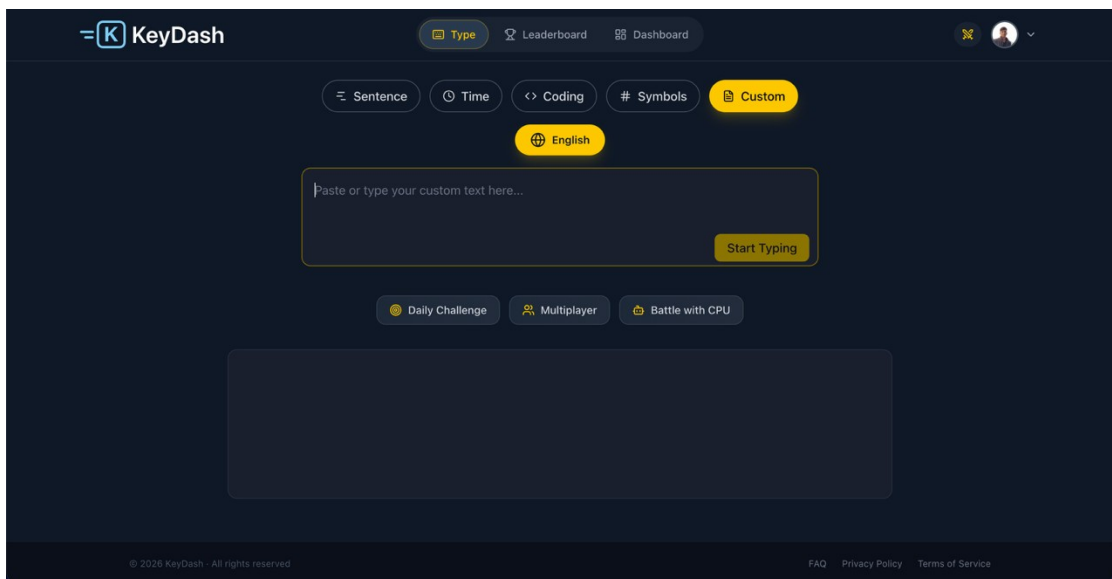
Appendix J: Battle with CPU



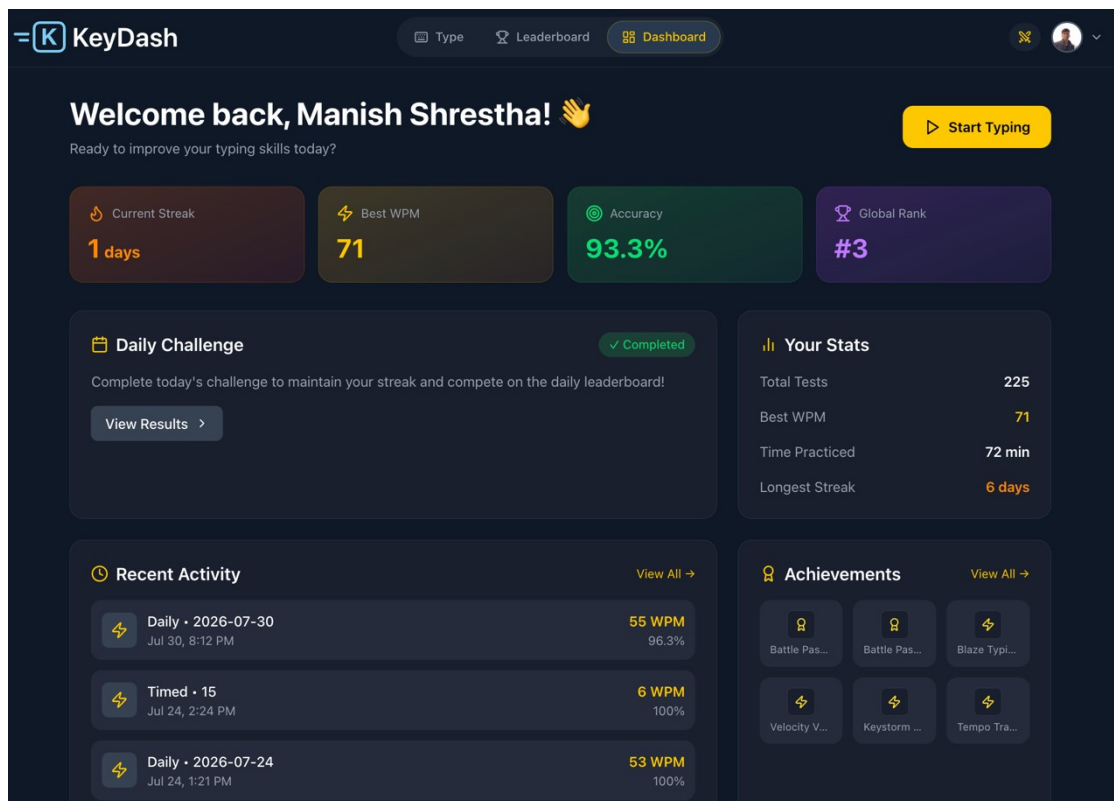
Appendix K: Coding Mode



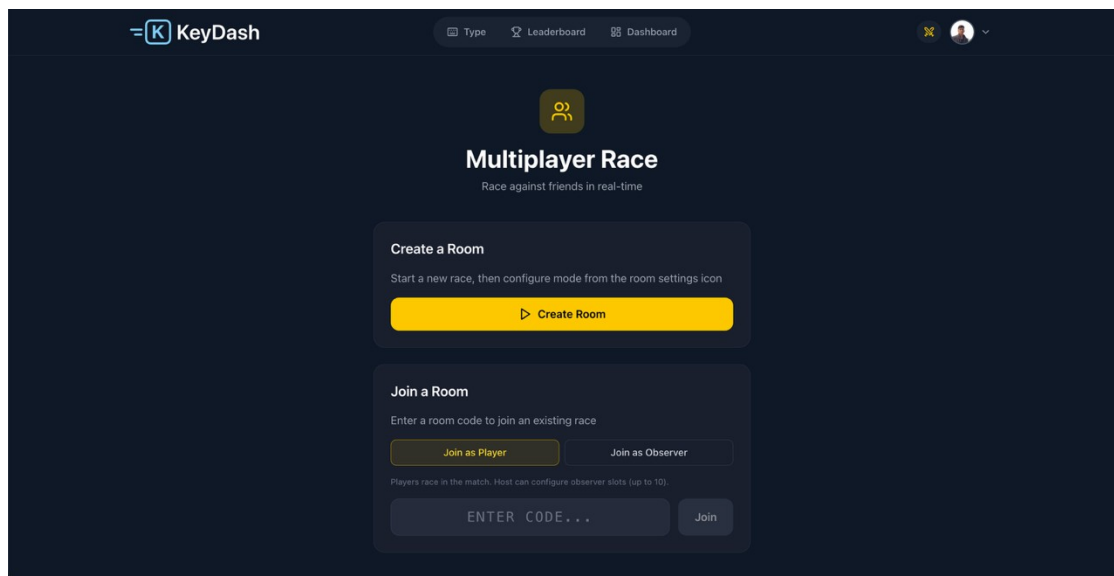
Appendix L: Symbols Mode



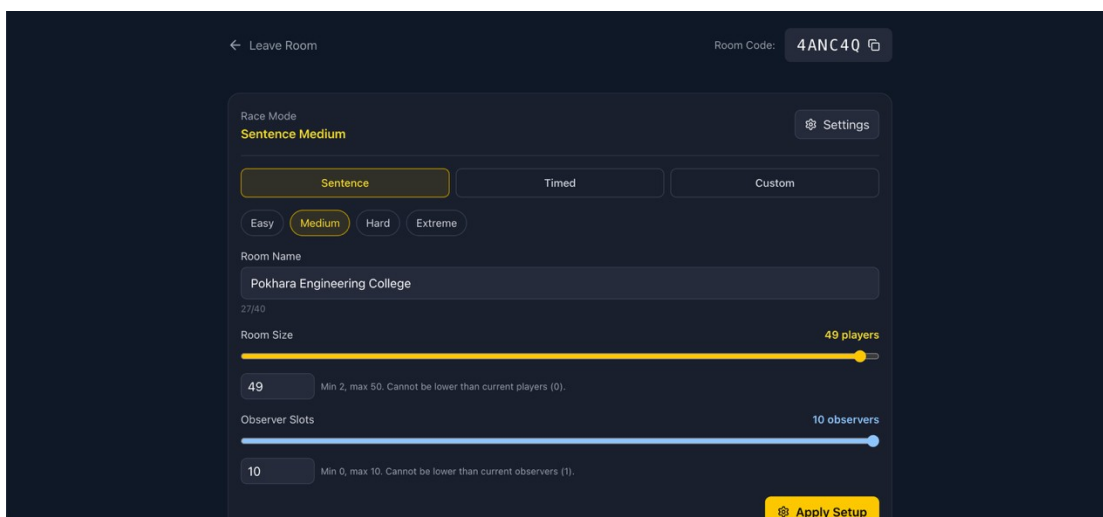
Appendix M: Custom Mode



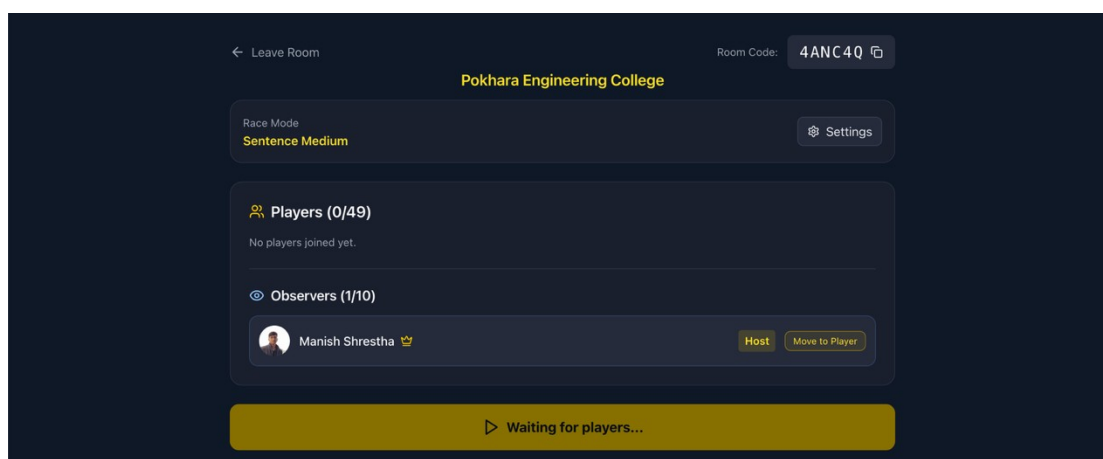
Appendix N: Dashboard Section



Appendix O: Multiplayer Race Mode



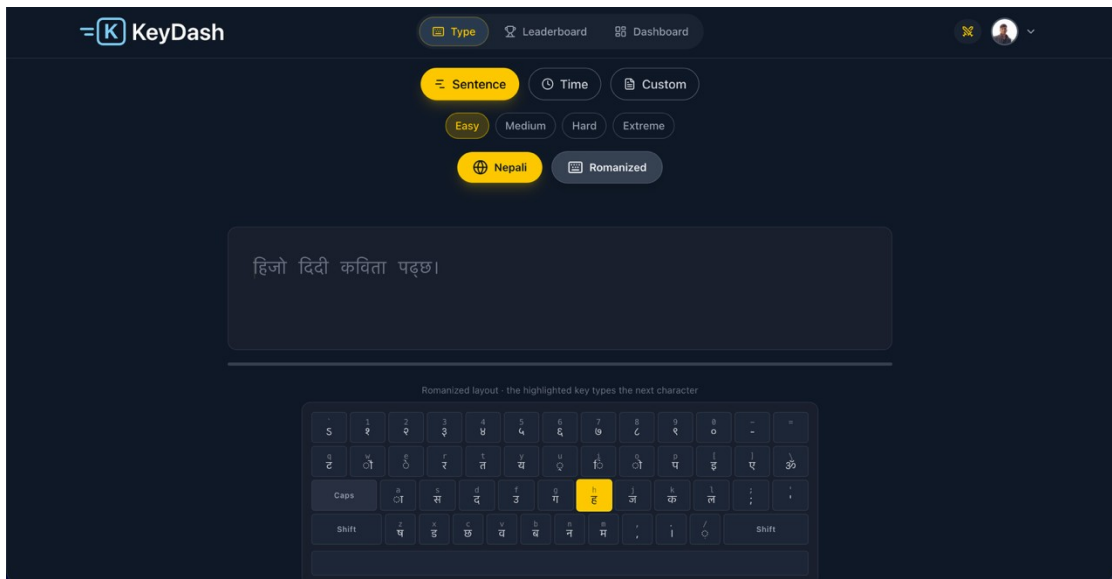
Appendix P: Multiplayer Race Setting



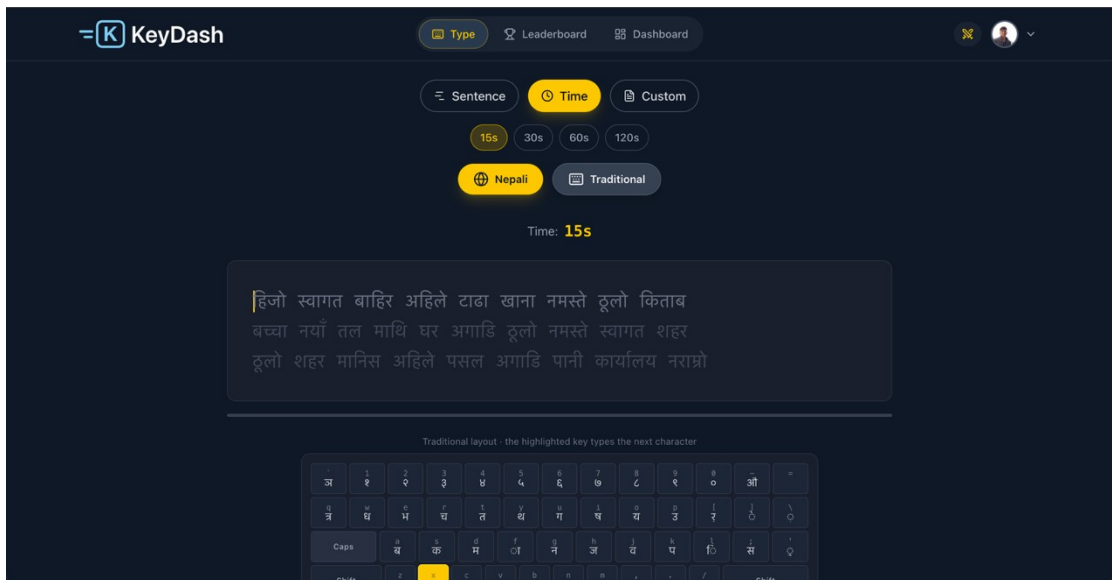
Appendix Q: Multiplayer Race Waiting Area

KeyDash Multiplayer Leaderboard			
Room: HWM7CY			
Mode: Sentence Medium			
Exported: 28/02/2026, 15:25:30			
#1	KeyDash	40 WPM	92.5% acc
#2	Manish Shrestha	38 WPM	98.1% acc

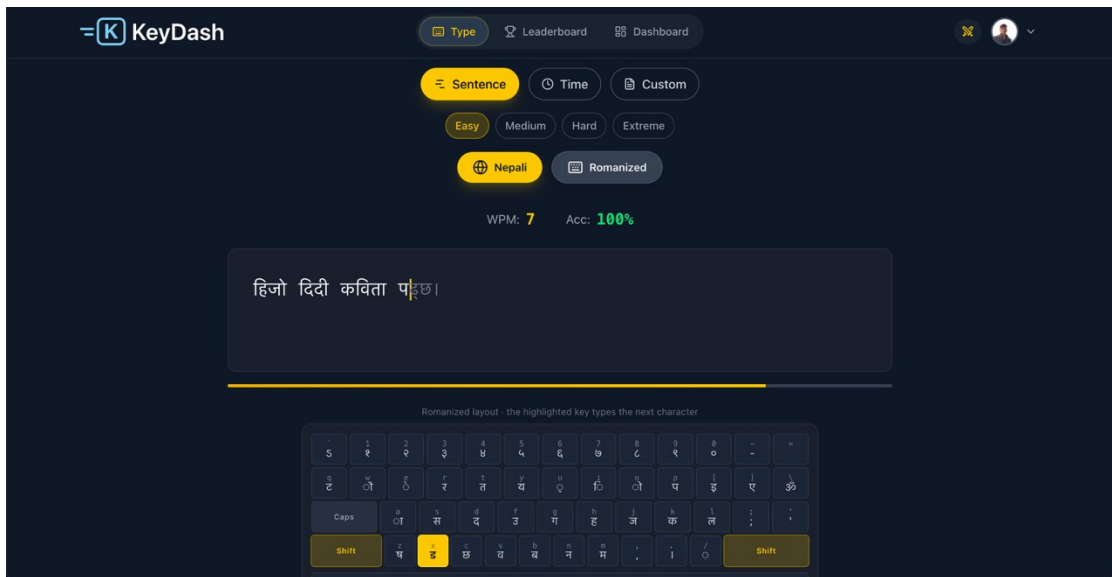
Appendix R: Multiplayer Race Result (From Observer View)



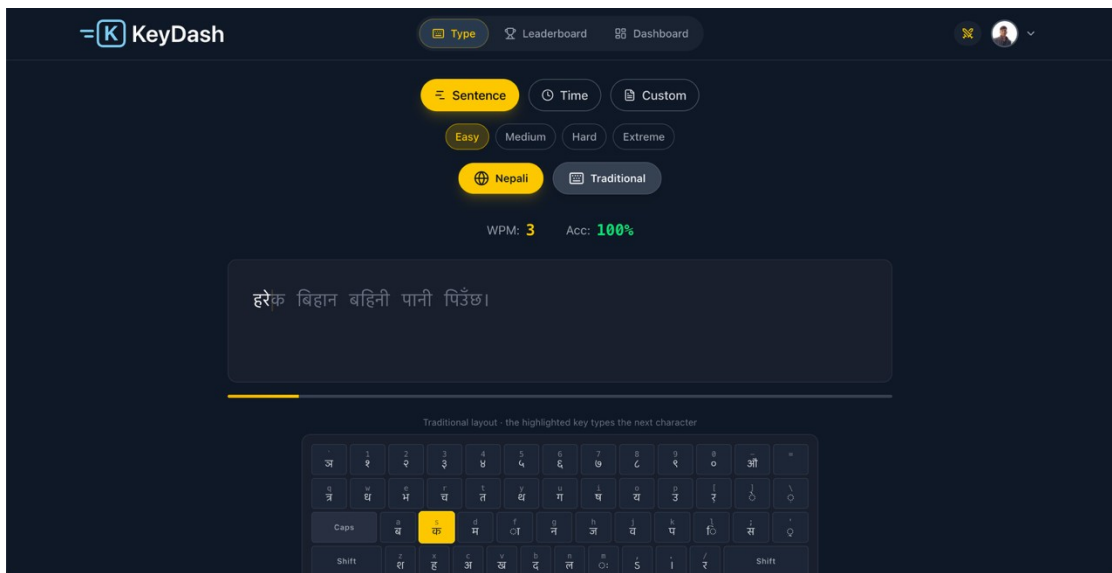
Appendix S: Nepali Language Sentence Mode



Appendix T: Nepali Language Time Mode



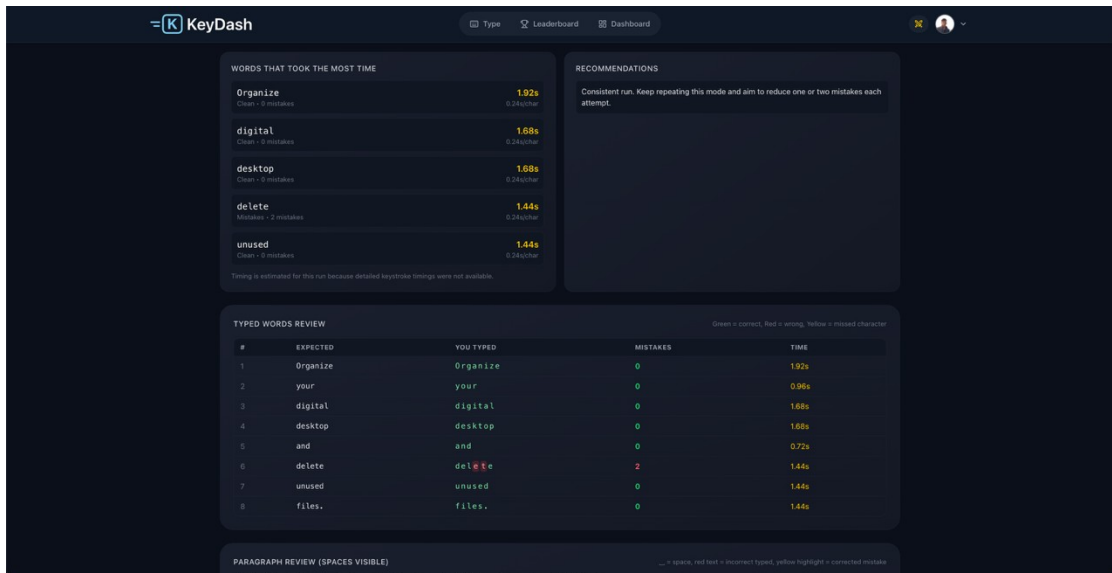
Appendix U: Nepali Language with Roman Keyboard



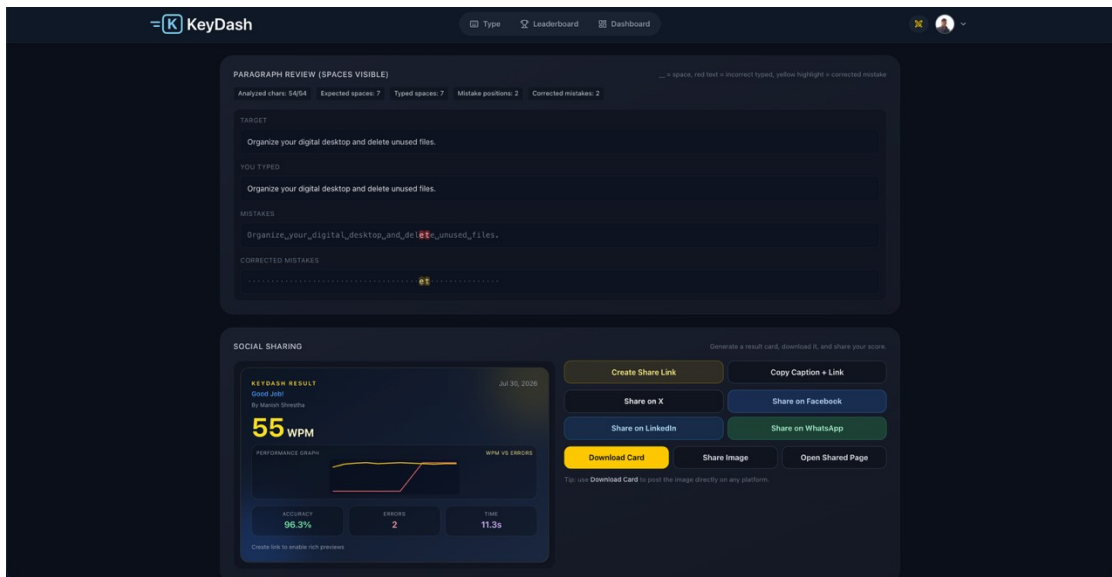
Appendix V: Nepali Language with Traditional Keyboard



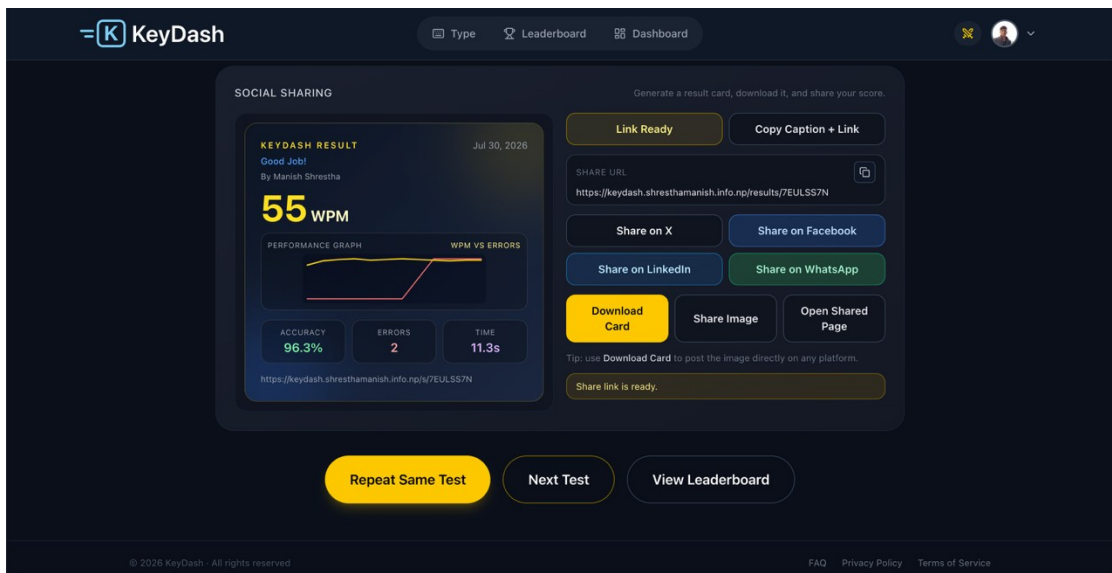
Appendix W: Result Page



Appendix X: Result Page Recommendations and Review



Appendix Y: Result Page Paragraph Review and Social Sharing



Appendix Z: Result Page Social Sharing Link Ready